



I want you to act as a **Senior Azure Data Engineer, Microsoft Fabric Architect, and Mentor** with real industry experience.

Design **one complete end-to-end enterprise Azure Data Engineering project using Microsoft Fabric (Free Trial)** that teaches me from **beginner to advanced**.

Before designing the project, analyze Microsoft Learn, Microsoft Fabric documentation, GitHub repositories, real-world projects, and industry best practices. Build a project that reflects how companies actually use Microsoft Fabric in production.

Assume I know only a few basic concepts. Teach me like I have just joined an MNC as a Junior Azure Data Engineer.

Choose one realistic business domain (E-commerce, Retail, Banking, Healthcare, Logistics, etc.) and use it throughout the project.

For every topic, always follow this learning approach:

1. Explain **what it is**.
2. Explain **why it is needed**.
3. Explain **what business problem it solves**.
4. Explain **why companies choose this approach instead of other approaches**, including comparisons where appropriate.
5. Explain **where it fits in the overall architecture**.
6. Give a simple real-world example or analogy.
7. Then implement it step by step in Microsoft Fabric.
8. Explain every setting, configuration, and line of code.
9. Explain common mistakes, production challenges, best practices, and interview points.

Build the project in logical phases, starting with planning and ending with a production-ready solution. Cover the complete data engineering lifecycle, including architecture, data ingestion, transformation, storage, modeling, orchestration, monitoring,

optimization, security, and reporting.

Use Microsoft Fabric services wherever appropriate, such as OneLake, Lakehouse, Data Factory, Data Pipelines, Dataflow Gen2, Notebooks, PySpark, SQL Endpoint, Warehouse, Semantic Model, Power BI, Monitoring, and other services available in the Fabric Free Trial. If a feature is unavailable in the trial, explain its purpose and demonstrate the closest practical alternative.

Do not skip steps or jump ahead. Complete one topic thoroughly before moving to the next. Use diagrams, tables, flowcharts, and architecture diagrams whenever they improve understanding.

Throughout the project, focus on helping me understand not only **how** to build the solution but also **why** each technology, design choice, and implementation approach is used. My goal is to gain practical industry-level Azure Data Engineering skills, understand real production scenarios, and become confident in Microsoft Fabric and Azure Data Engineering.

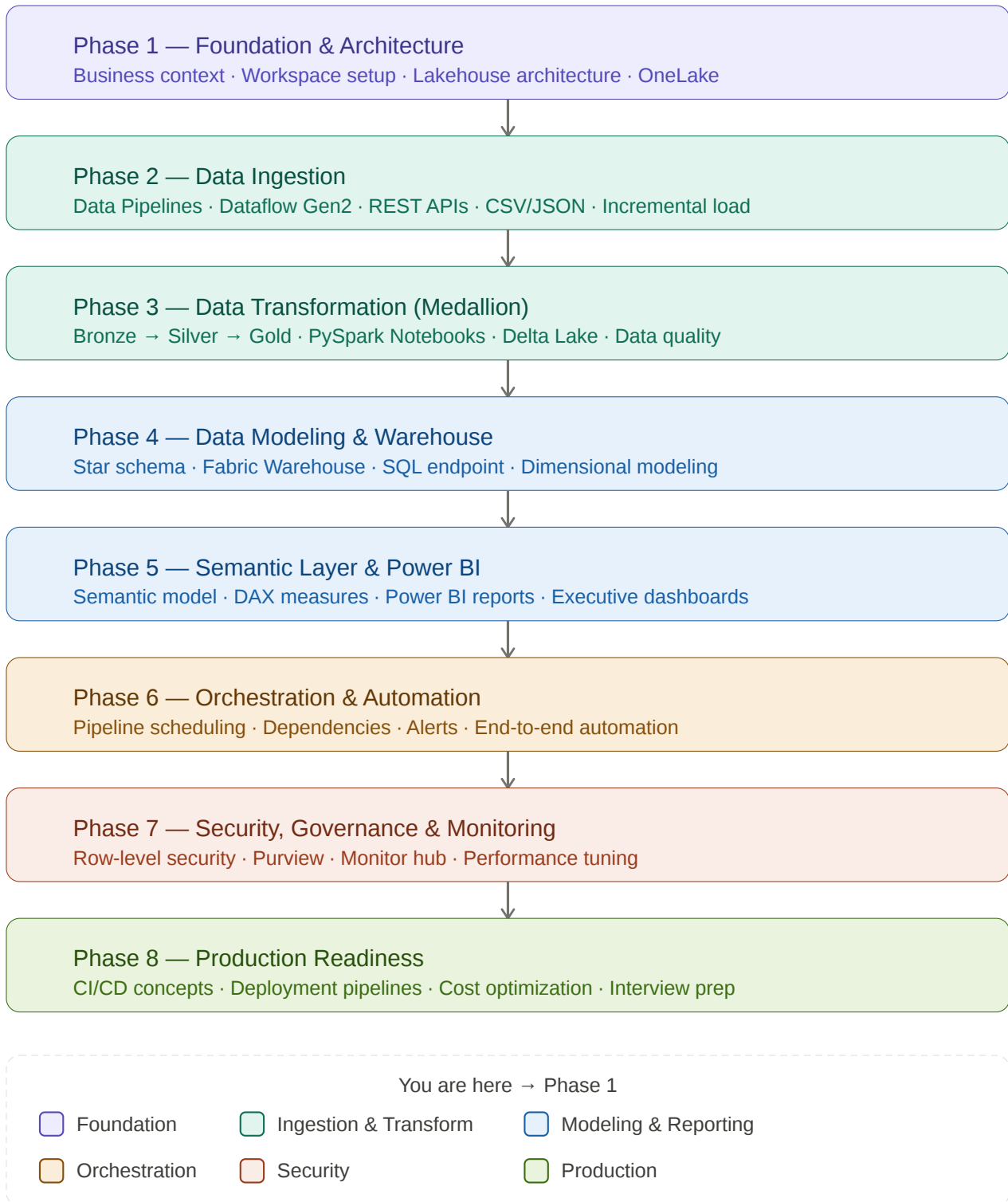


What an ambitious and exciting learning journey you're embarking on! Let me design a comprehensive, production-grade Microsoft Fabric project for you. I'll start with the foundation — the big picture, architecture, and project context — before we build anything.

Let me first render the overall project roadmap and architecture so you always know where you are in the journey.

RetailPulse — end-to-end Microsoft Fabric project

E-commerce analytics · Beginner → Production-ready



Phase 1 — Foundation & Architecture

1.1 The Business Domain: RetailPulse E-commerce

What is it?

We're building a complete data engineering platform for a fictional company called **RetailPulse** — a mid-sized Indian e-commerce company (think a smaller Flipkart or Myntra) that sells electronics, apparel, and home goods across India. They have 500,000+ customers, process 10,000–50,000 orders per day, and operate warehouses in Mumbai, Delhi, Bangalore, and Hyderabad.

Why this domain?

E-commerce is the perfect teaching domain because it generates every type of data a real data engineer encounters:

Data Type	Source	Real-World Challenge
Transactional	Orders, payments, returns	High volume, must be accurate
Behavioral	Clicks, searches, cart events	Semi-structured JSON, high velocity
Reference/Master	Products, customers, sellers	Slowly changing, needs history
Operational	Inventory, logistics, delivery	Near real-time updates
Financial	Revenue, margins, refunds	Regulatory accuracy required

The Business Problem We're Solving

RetailPulse's leadership team is asking questions their current systems can't answer:

- "Why did our revenue drop 12% last Tuesday in Bangalore?"
- "Which product categories have the highest return rates, and why?"
- "Which customers are about to churn?"
- "Are our warehouse SLAs being met?"

Right now, the data team spends 3 days manually extracting data from 5 different systems, pasting it into Excel, and emailing reports. The CEO makes decisions based on week-old data. This is the exact scenario that drives companies to build a modern data platform.

1.2 What is Microsoft Fabric?

What is it?

Microsoft Fabric is Microsoft's **unified analytics platform** launched in 2023. Think of it as one single product that combines everything a data team needs: data ingestion, storage, transformation, modeling, and reporting — all inside a single workspace, billed together, sharing one storage layer called OneLake.

Why was it created?

Before Fabric, a company using Azure would need to stitch together many separate products:

Old Approach (Azure Patchwork)	New Approach (Microsoft Fabric)
Azure Data Factory (ingestion)	Data Factory (built-in)
Azure Data Lake Storage Gen2	OneLake (unified)
Azure Databricks or Synapse Spark	Spark Notebooks (built-in)
Azure Synapse Analytics (warehouse)	Fabric Warehouse (built-in)
Power BI (reporting)	Power BI (native, deeply integrated)
Separate billing, separate auth, data copying between services	One bill, one identity, zero data copying

This is a massive shift. In the old world, just getting data from your data lake into your reporting tool required copying data (expensive, slow, error-prone). In Fabric, everything reads from the same OneLake storage.

Real-World Analogy

Think of the old Azure approach like a kitchen where you have separate appliances from different brands – a German fridge, a Japanese microwave, an Italian oven – that don't communicate. You have to manually move food between them. Microsoft Fabric is like a modern integrated kitchen where everything is connected: the fridge tells the oven what's inside, the microwave knows your preferences, and you control everything from one panel.

Why do companies choose Fabric over alternatives?

Factor	Microsoft Fabric	AWS (Glue + Redshift + QuickSight)	Databricks + dbt + Tableau
Integration effort	Minimal – all native	High – multiple services to wire	Medium – good ecosystem
Cost model	Capacity-based (predictable)	Pay per use (can spike)	License + cloud costs
Power BI integration	Native, deep	3rd party	3rd party
Setup time for beginners	Low	High	Medium
Enterprise Microsoft ecosystem fit	Perfect (Azure AD, M365)	Requires extra work	Requires extra work
Open format (Delta/Parquet)	Yes	Partial	Yes

Companies already using Microsoft 365, Azure Active Directory, or Power BI find Fabric extremely compelling because there is essentially zero new identity/security infrastructure to set up.

1.3 Understanding OneLake — The Foundation of Everything

What is OneLake?

OneLake is the **single storage layer** that underlies all of Microsoft Fabric. Every piece of data in Fabric — whether it came from a pipeline, a notebook, a dataflow, or a warehouse — lives in OneLake. It's built on top of Azure Data Lake Storage Gen2 under the hood, but you never interact with ADLS directly.

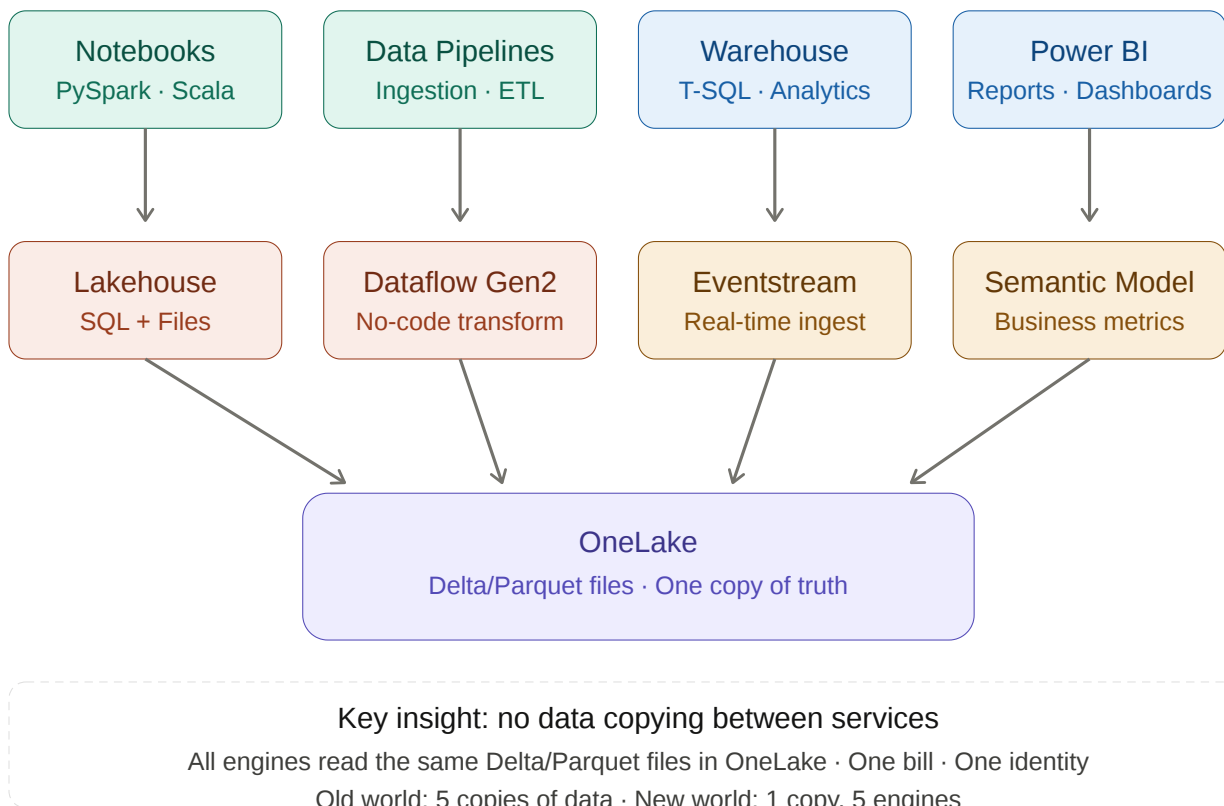
Why is this revolutionary?

In traditional architectures, data engineers spent enormous time on **data movement**: copying data from a data lake to a data warehouse, then copying again to a reporting database. Each copy introduced:

- Latency (your data is always "behind")
- Cost (storage and compute for each move)
- Inconsistency (the warehouse says revenue is ₹10 crore, the lake says ₹10.2 crore — which is right?)

OneLake eliminates all of this. The same file that your Spark notebook wrote is the same file your SQL query reads and the same file Power BI reports on. There is only one copy of truth.

OneLake — single storage, multiple engines



1.4 The Medallion Architecture — Our Core Design Pattern

What is it?

The Medallion Architecture is the most widely adopted data organization pattern in modern data engineering. It organizes data into three progressive quality layers, named after precious metals:

- **Bronze** — Raw data exactly as it arrived. Nothing changed. Think of it as your permanent backup.
- **Silver** — Cleaned, validated, and standardized data. Duplicates removed, data types corrected, business rules applied.
- **Gold** — Business-ready, aggregated, modeled data ready for reporting and analytics.

Why do companies use it?

Interview Point: This is one of the most common data engineering interview questions — "Walk me through the Medallion Architecture and why you'd use it."

The Medallion pattern solves the fundamental tension in data engineering: raw data is ugly and unreliable, but you must preserve it. Here's why each layer matters:

Bronze (Raw): If you transform data immediately on arrival and something goes wrong (a bug in your logic, a business rule that changes), you have lost the original. Bronze is your insurance policy. Companies have been fined millions in regulatory audits because they couldn't prove what their raw data looked like at a specific time.

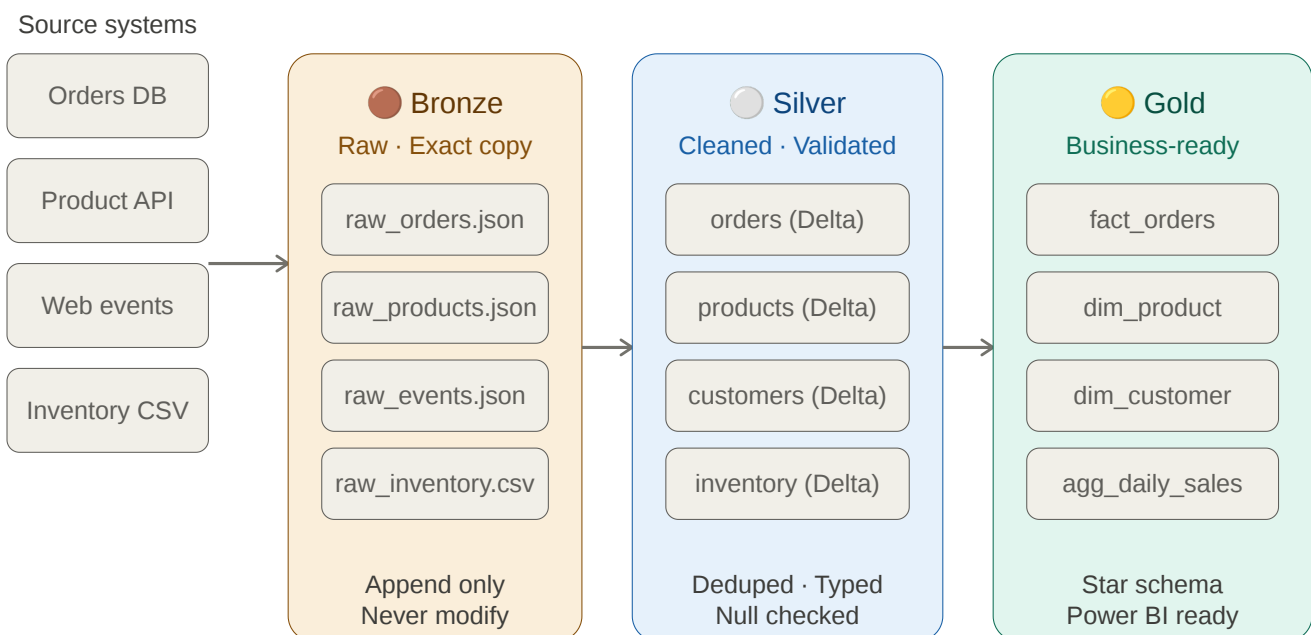
Silver (Cleaned): Analytics teams need consistent data. If "Mumbai" is spelled "Mumbai", "mumbai", "MUM", and "Mmbai" across your dataset, every downstream report gives different results. Silver fixes this once and all Gold tables inherit the fix.

Gold (Aggregated): Power BI reports hitting raw data with billions of rows are impossibly slow. Gold pre-computes expensive aggregations so dashboards load in under 2 seconds.

Where does this fit in our architecture?

Every Lakehouse in our project maps to a Medallion layer. We will have three Lakehouses (or folders within one Lakehouse):

- `bronze_lakehouse` — raw data, append-only
- `silver_lakehouse` — cleaned, Delta tables
- `gold_lakehouse` — dimensional model, star schema



What changes layer by layer

Bronze: "order_date": "2024-01-15T09:32:11Z", "amount": "1299.00", "city": "mmbai"

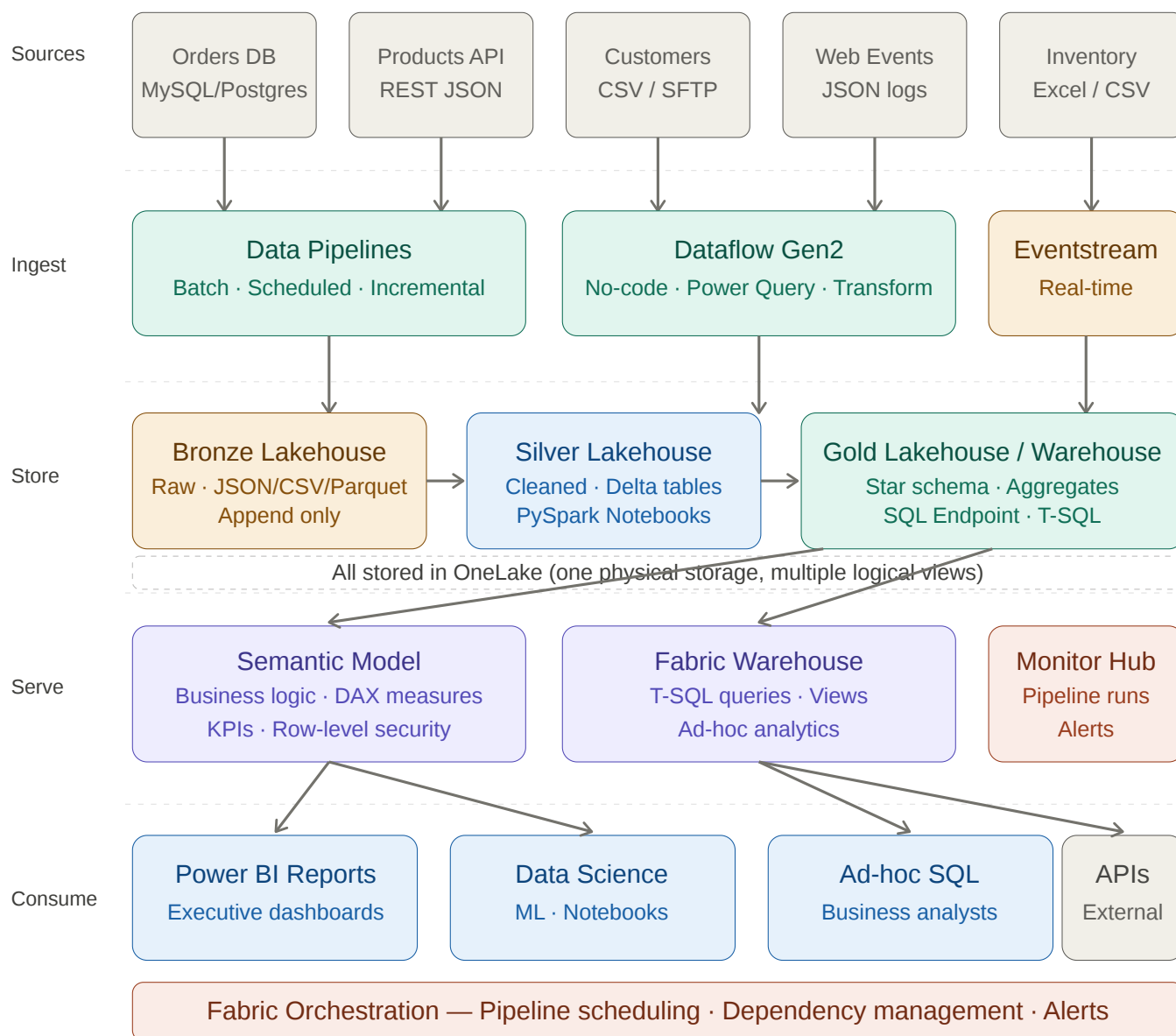
Silver: order_date: 2024-01-15 (date), amount: 1299.00 (decimal), city: "Mumbai" (corrected)

Gold: date_key: 20240115, total_revenue: ₹45,23,100, city_name: "Mumbai", orders: 342

(pre-aggregated per day, per city — Power BI reads this in milliseconds)

1.5 Full End-to-End Architecture for RetailPulse

Before we write a single line of code, let's understand the complete blueprint. Senior engineers always design before they build.



1.6 Setting Up Your Microsoft Fabric Free Trial – Step by Step

What you need

- A work or school email address (free Gmail/Hotmail will not work – you need an organizational email, OR you create a free Microsoft 365 Developer account)
- A web browser (Edge or Chrome recommended)

Step 1 – Get an organizational email (if needed)

If you don't have a work email, sign up for the **Microsoft 365 Developer Program** at developer.microsoft.com/en-us/microsoft-365/dev-program . This gives you a free dev tenant with an org email like yourname@yourtenant.onmicrosoft.com . This is what most learners use.

Step 2 — Activate Fabric Free Trial

1. Go to app.fabric.microsoft.com
2. Sign in with your organizational email
3. Click your account icon (top right) → "Start trial"
4. The trial gives you 60 days of Fabric F64 capacity (worth ~\$6,000/month)
5. Click "Activate" — this takes 2–5 minutes

Important: The trial gives you real production-grade compute. Everything we build will work exactly as it would in a company's production environment.

Step 3 — Create Your Fabric Workspace

A **Workspace** in Fabric is like a project folder. It contains all your Lakehouses, Pipelines, Notebooks, and Reports. Think of it as a GitHub repository for your entire data platform.

1. In Fabric, click "Workspaces" in the left sidebar
2. Click "+ New workspace"
3. Name it: [RetailPulse_DataPlatform](#)
4. Description: [End-to-end e-commerce analytics platform](#)
5. Under "Advanced" → License mode: select **Trial** (or Fabric capacity if you see it)
6. Click "Apply"

Why workspace naming matters: In production, companies typically have separate workspaces per environment ([retailpulse_dev](#) , [retailpulse_staging](#) , [retailpulse_prod](#)). Each has different access levels. Your workspace is your dev environment.

Step 4 — Create Your Three Lakehouses

Within your workspace, create three Lakehouses for our Medallion architecture:

Creating Bronze Lakehouse:

1. Inside your workspace, click "+ New item"
2. Select **Lakehouse**
3. Name: [bronze_lakehouse](#)
4. Click "Create"

Wait — what just happened? When you create a Lakehouse, Fabric automatically:

- Creates a folder structure in OneLake
- Provisions a SQL analytics endpoint (so you can query the Lakehouse with T-SQL)
- Sets up Delta Lake format support
- Creates a default semantic model

Repeat for Silver and Gold:

- silver_lakehouse
- gold_lakehouse

Step 5 – Understand the Lakehouse Interface

Click on bronze_lakehouse . You'll see:

Section	What it is
Tables	Delta tables you can query with SQL
Files	Raw files (JSON, CSV, Parquet) in OneLake
SQL analytics endpoint	T-SQL query interface against your Delta tables
Explorer panel	Browse OneLake file hierarchy

Key concept – Tables vs Files:

- **Files** folder: unstructured raw storage. You dump JSON/CSV here. Think of it like a folder on your hard drive.
- **Tables** section: Delta Lake tables with schema, query capability, and versioning. Once you register a table here, it's queryable via SQL.

In our project: Bronze Lakehouse will mostly use Files (raw dumps). Silver and Gold Lakehouses will use Tables (Delta format, SQL-queryable).

1.7 The Data We'll Work With

Here are the datasets for RetailPulse. We'll simulate these with synthetic data generators in Phase 2.

Dataset	Records	Update Frequency	Format
Orders	~10K/day	Real-time → batch	JSON
Order items	~25K/day	With orders	JSON (nested)
Customers	500K total, ~200 new/day	Daily	CSV
Products	50K products, ~50 updates/day	Daily	REST API (JSON)

Dataset	Records	Update Frequency	Format
Inventory	50K SKUs	Hourly	CSV
Web events	~500K events/day	Near real-time	JSON logs
Payments	~10K/day	With orders	JSON

This is intentionally messy, just like real data:

- Orders have **nullable fields** (some orders have no discount code)
- Customer cities are **inconsistently spelled** (Mumbai, mumbai, MUMBAI, MUM)
- Prices are sometimes **strings** ("1299.00") and sometimes **numbers** (1299.00)
- Some orders have **missing product IDs** (referential integrity violations)
- Web events are **deeply nested JSON**

1.8 Key Concepts Checklist Before Phase 2

Before moving to Phase 2, make sure you can answer these:

Conceptual:

- What problem does OneLake solve compared to traditional data lakes?
- Why do we have three Lakehouses instead of one?
- What goes in the Files section vs Tables section of a Lakehouse?

Practical (do these now):

- Have you activated your Fabric trial?
- Have you created the `RetailPulse_DataPlatform` workspace?
- Have you created `bronze_lakehouse` , `silver_lakehouse` , and `gold_lakehouse` ?

Interview Questions You Can Now Answer:

1. "Explain the Medallion Architecture" — You know what Bronze/Silver/Gold are, what each stores, and why.
2. "What is Microsoft Fabric and how does it differ from Azure Synapse?" — You know Fabric is unified (one storage, one identity, integrated billing) vs Synapse was a collection of loosely coupled services.
3. "What is OneLake?" — Single storage layer, no data copying, all engines read the same Delta/Parquet files.

4. "Why would a company choose Fabric?" — Already on Microsoft stack, want unified billing and governance, need Power BI integration, want to reduce data movement.
-

Common Mistakes at This Stage

Mistake 1 — Skipping workspace planning. Many beginners create random items without a workspace structure. Always plan your workspace → Lakehouse → naming conventions before creating anything. In production, renaming things after the fact is painful.

Mistake 2 — Using personal email for Fabric trial. Microsoft Fabric requires an organizational email. Use the M365 Developer Program if you don't have one.

Mistake 3 — Confusing Lakehouse with Warehouse. A Lakehouse stores files + Delta tables, supports both file-level access and SQL. A Warehouse is a pure SQL T-SQL warehouse with no file access. We use both — Lakehouse for transformation work, Warehouse for advanced SQL analytics in Gold layer.

Mistake 4 — Thinking you need to set up ADLS separately. OneLake IS your storage. You never create an ADLS account — Fabric handles this automatically.

What's Coming in Phase 2

Now that your workspace and three Lakehouses exist, Phase 2 will teach you to **pull data into Bronze** using:

- **Data Pipelines** — the visual, no-code way to copy data from any source
- **Dataflow Gen2** — Power Query in the cloud, for transformation during ingestion
- **Notebooks** — Python/PySpark for when you need programmatic control

We'll build the first pipeline that ingests synthetic order data into `bronze_lakehouse`, configure incremental loading (a critical production skill), handle errors, and monitor the run.

Ready to move to **Phase 2 — Data Ingestion**? Just say "continue to Phase 2" and we'll start building the actual pipelines. Or if you have questions about anything in Phase 1 — the architecture, workspace setup, or Medallion design decisions — ask now before we build on top of it.

2026-06-14 00:52:01 You Asked 

yes continue good and explain all steps also with guide .

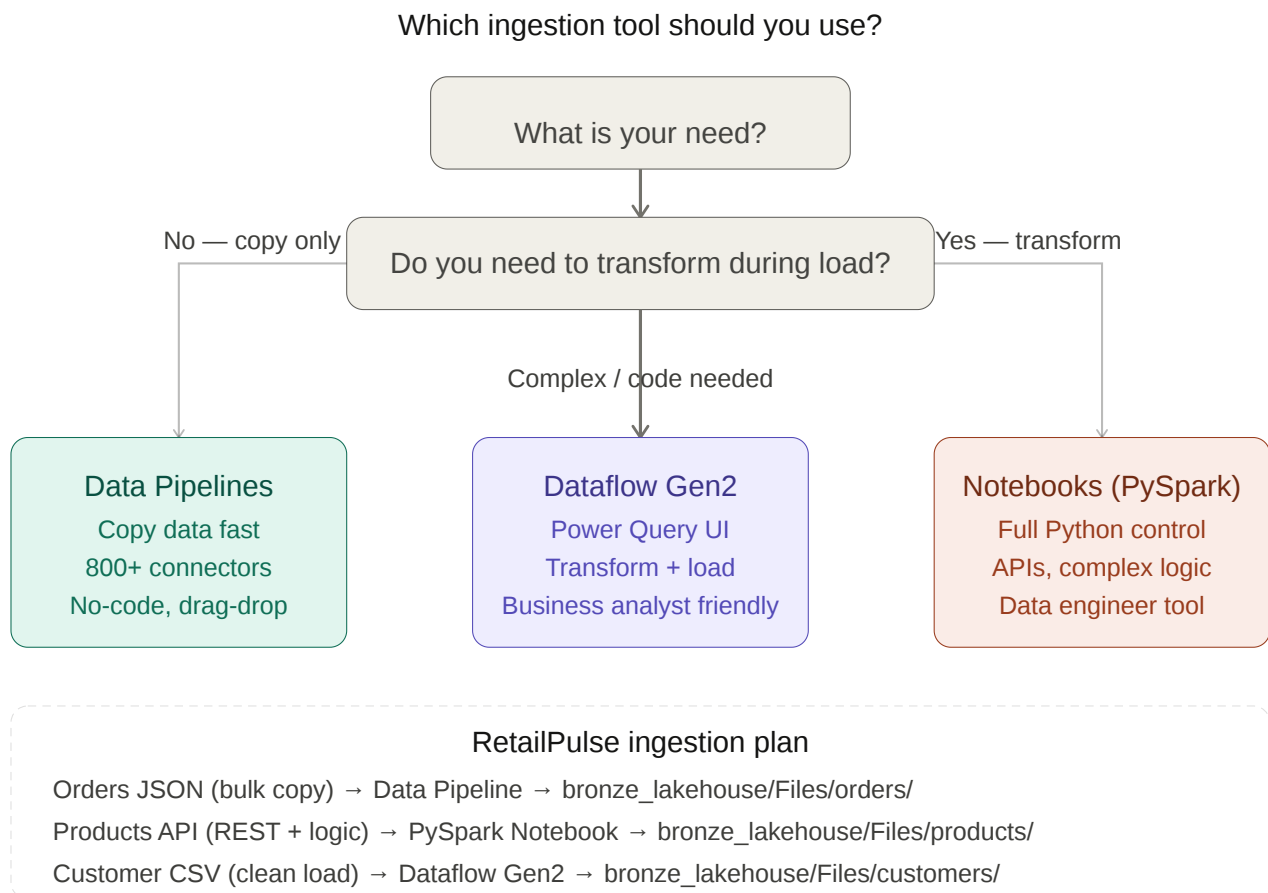
Let me build Phase 2 completely for you — ingestion is where real data engineering begins.

Phase 2 — Data Ingestion

This is where your data platform comes to life. Ingestion is the very first step in any data pipeline — pulling raw data from source systems and landing it safely in your Bronze Lakehouse. Let's understand every tool, every concept, and every click.

2.1 The Three Ingestion Tools in Fabric — When to Use What

Before touching Fabric, you need to understand the three main ways to bring data in, because choosing the wrong tool wastes hours.



Now let's build each one, step by step, with every setting explained.

2.2 Understanding Delta Lake Before We Write Any Data

What is Delta Lake?

Delta Lake is the file format that sits underneath all Fabric Lakehouse tables. When you write a "table" in Fabric, it actually creates Parquet files plus a `_delta_log` folder containing a transaction

log. This combination is what makes it a Delta table.

Why does it matter to you as a data engineer?

Feature	What it means practically
ACID transactions	Two pipelines writing simultaneously won't corrupt data
Time travel	<code>SELECT * FROM orders VERSION AS OF 5</code> — query data as it was 5 versions ago
Schema enforcement	Writing a column with wrong data type throws an error, not silent corruption
Upserts (MERGE)	Update existing rows + insert new rows in one atomic operation
Partition pruning	Queries only scan the date partition you need, not the whole table

Think of Delta Lake as "Git for data" — every write is a commit, you can roll back, and you always know exactly what changed and when. This is non-negotiable in production data engineering.

2.3 Ingestion Method 1 — Data Pipelines (Copy Activity)

What is a Data Pipeline?

A Data Pipeline in Microsoft Fabric is a visual, no-code workflow builder. You drag activities onto a canvas and connect them. The most important activity is the **Copy Data** activity — it can connect to 800+ data sources (databases, APIs, file storage, SaaS apps) and copy data to your Lakehouse with zero code.

Why use it instead of writing code?

When your job is simply "move data from A to B" — no complex transformation, just faithful copying — a pipeline does it faster, more reliably, and with built-in monitoring. Every pipeline run is automatically logged. If it fails at 3am, you get an alert. You don't get that for free with a Python script.

Real-world analogy

A Data Pipeline is like a conveyor belt in a factory. You define the source (where raw materials come from), the destination (where finished goods go), and the speed. The factory floor manager (Fabric) monitors it, restarts it if it breaks, and logs every batch that passed through.

Step-by-Step: Create Your First Data Pipeline

We'll build a pipeline that generates synthetic order data and lands it in `bronze_lakehouse`. Since we don't have a real database to connect to in a free trial, we'll use the **HTTP connector** to pull data from a public JSON API that simulates orders.

Step 1 — Open Data Factory in Fabric

1. Go to your `RetailPulse_DataPlatform` workspace
2. In the bottom-left corner, click the **Experience switcher** (the icon that looks like a grid or shows "Home")
3. Select **Data Factory** from the menu
4. You're now in the Data Factory experience inside Fabric

Why a separate "experience"? Fabric has multiple "personas" — Data Factory, Data Engineering, Data Science, Data Warehouse, Real-Time Intelligence, Power BI. They're all the same product, same workspace, but each persona shows you the relevant tools for that role. You'll switch between them throughout this project.

Step 2 — Create a New Data Pipeline

1. Click **+ New item**
2. Select **Data pipeline**
3. Name it: `pl_ingest_orders_bronze`

Naming convention explained: `pl_` = pipeline, `ingest` = what it does, `orders` = which data, `bronze` = which layer. In production, consistent naming lets you find any artifact instantly. Senior engineers enforce naming standards team-wide.

4. Click **Create**

You now see the Pipeline canvas — a blank white canvas with a toolbar at the top.

Step 3 — Add a Copy Data Activity

1. In the canvas toolbar, click **Copy data** (or drag it from the Activities panel on the left)
2. A blue box labeled "Copy data1" appears on the canvas
3. Click on it to select it — a properties panel appears at the bottom

The pipeline canvas has three main areas: the canvas (where you build visually), the properties panel (where you configure each activity), and the toolbar (where you add activities). Everything you do is JSON under the hood — Fabric is just giving you a visual editor for it.

Step 4 — Configure the Source (Where data comes from)

Click the **Source** tab in the bottom properties panel.

1. Click **+ New** next to "Source dataset"
2. In the search box, type `HTTP` and select **HTTP**
3. Click **Continue**
4. You're now creating a **Linked Service** — this is a saved connection definition

What is a Linked Service? Think of it as a saved bookmark + credentials for a data source. You create it once, reuse it in many pipelines. If the API URL changes, you update one Linked Service and all pipelines using it automatically get the update.

Configure the HTTP Linked Service:

- Name: `ls_http_mock_orders`
- Base URL:
`https://raw.githubusercontent.com/microsoft/fabric-samples/main/docs-samples/data-engineering/`
(we'll use a public dataset)
- Authentication type: `Anonymous`
- Click **Create**

For the actual mock data, we'll use a cleverly constructed approach. Since we need realistic RetailPulse data, we'll first upload synthetic CSV files to the Lakehouse Files section and then use the pipeline to process them. Here's exactly what to do:

Step 4b — Upload Synthetic Data to Lakehouse (Practical for Free Trial)

Since connecting to live external databases requires credentials we don't have, here's the production-realistic approach: upload synthetic source files to a "landing zone" folder in bronze, then build a pipeline that moves and processes them. This is exactly how companies with SFTP drops, email attachments, and vendor file deliveries work.

First, create synthetic data files. Open a Notebook in Fabric to generate the data:

1. Go back to your workspace
2. Click **+ New item** → **Notebook**
3. Name it: `nb_generate_synthetic_data`

Copy and run this code cell by cell:

```
python

# =====
# CELL 1 - Install libraries and import
# =====
# Fabric Notebooks have PySpark pre-installed
# For data generation we use the built-in random + faker approach
```

```

import random
import json
import csv
import io
from datetime import datetime, timedelta
from pyspark.sql import SparkSession
from pyspark.sql.types import *
import pyspark.sql.functions as F

# SparkSession is already available in Fabric notebooks as 'spark'
print("Spark version:", spark.version)
print("Ready to generate data!")

```

What is SparkSession? It's the entry point to all Spark functionality. In Fabric Notebooks, it's automatically created for you as `spark`. You never need to call `SparkSession.builder.getOrCreate()` — Fabric does it. This is one of the key conveniences of Fabric over raw Databricks.

python

```

# =====
# CELL 2 – Generate synthetic Orders data
# =====

# Configuration – mirrors real RetailPulse business
CITIES = ["Mumbai", "Delhi", "Bangalore", "Hyderabad", "Chennai",
          "Pune", "Kolkata", "Ahmedabad", "Jaipur", "Surat"]

# Intentional data quality issues (just like real data!)
CITY_VARIANTS = {
    "Mumbai": ["Mumbai", "mumbai", "MUMBAI", "MUM", "Mmbai"],
    "Delhi": ["Delhi", "delhi", "New Delhi", "DELHI", "NCR"],
    "Bangalore": ["Bangalore", "Bengaluru", "bangalore", "BLR"],
}

CATEGORIES = ["Electronics", "Apparel", "Home & Kitchen",
              "Books", "Sports", "Beauty", "Toys", "Automotive"]

PAYMENT_METHODS = ["UPI", "Credit Card", "Debit Card",
                   "Net Banking", "Cash on Delivery", "Wallet"]

STATUSES = ["placed", "confirmed", "shipped", "delivered",
            "cancelled", "returned"]

STATUS_WEIGHTS = [0.05, 0.10, 0.20, 0.45, 0.12, 0.08]

```

```

def random_city_messy():
    """Returns a city name with realistic typos/variants"""
    city = random.choice(CITIES)
    if city in CITY_VARIANTS and random.random() < 0.25:
        return random.choice(CITY_VARIANTS[city]) # 25% chance of messy spelling
    return city

def generate_order(order_id, date):
    """Generate one realistic order record"""
    num_items = random.randint(1, 5)
    items = []
    order_total = 0

    for i in range(num_items):
        price = round(random.uniform(99, 49999), 2)
        qty = random.randint(1, 3)
        discount = round(random.uniform(0, 0.30), 2) if random.random() > 0.4 else 0
        item_total = round(price * qty * (1 - discount), 2)
        order_total += item_total

        items.append({
            "item_id": f"ITEM{random.randint(10000, 99999)}",
            "product_id": f"PROD{random.randint(1000, 9999)}",
            "product_name": f"Product {random.randint(100, 999)}",
            "category": random.choice(CATEGORIES),
            "unit_price": price,
            # Intentional issue: price sometimes as string
            "unit_price_str": str(price) if random.random() < 0.15 else price,
            "quantity": qty,
            "discount_pct": discount,
            "item_total": item_total
        })

    # Intentional issue: sometimes customer_id is null
    customer_id = f"CUST{random.randint(10000, 99999)}" \
        if random.random() > 0.02 else None

    return {
        "order_id": f"ORD{order_id:07d}",
        "order_date": date.strftime("%Y-%m-%dT%H:%M:%S"),
        "customer_id": customer_id,
        "city": random_city_messy(), # messy city names!
        "state": random.choice(["MH", "DL", "KA", "TS", "TN", "GJ"]),
        "payment_method": random.choice(PAYMENT_METHODS),
        "order_status": random.choices(STATUSES, weights=STATUS_WEIGHTS)[0],
        "order_total": round(order_total, 2),
        # Intentional issue: some orders missing discount_code field
        "discount_code": f"SAVE{random.randint(10,30)}" \

```

```

        if random.random() > 0.6 else None,
        "items": items,
        "ingestion_timestamp": datetime.now().isoformat()
    }

# Generate 30 days of orders (300 orders/day = 9,000 total)
print("Generating orders...")
all_orders = []
start_date = datetime(2024, 1, 1)

for day in range(30):
    current_date = start_date + timedelta(days=day)
    # Vary order volume by day (weekends have more orders)
    daily_volume = 400 if current_date.weekday() >= 5 else 280

    for i in range(daily_volume):
        order_num = day * 500 + i + 1
        order_time = current_date + timedelta(
            hours=random.randint(0, 23),
            minutes=random.randint(0, 59),
            seconds=random.randint(0, 59)
        )
        all_orders.append(generate_order(order_num, order_time))

print(f"Generated {len(all_orders):,} orders")
print("Sample order:")
print(json.dumps(all_orders[0], indent=2))

```

python

```

# =====
# CELL 3 – Save orders to Bronze Lakehouse
# =====

# Convert to Spark DataFrame then save as JSON files
# We save as JSON (not Delta) because Bronze = raw format

orders_flat = []
for order in all_orders:
    # Flatten the nested structure for the main order record
    flat = {k: v for k, v in order.items() if k != 'items'}
    flat['items_json'] = json.dumps(order['items']) # Keep items as JSON string
    flat['item_count'] = len(order['items'])
    orders_flat.append(flat)

# Create a Spark DataFrame
schema = StructType([

```

```

StructField("order_id", StringType(), False),
StructField("order_date", StringType(), True),
StructField("customer_id", StringType(), True), # nullable!
StructField("city", StringType(), True),
StructField("state", StringType(), True),
StructField("payment_method", StringType(), True),
StructField("order_status", StringType(), True),
StructField("order_total", DoubleType(), True),
StructField("discount_code", StringType(), True), # nullable!
StructField("items_json", StringType(), True),
StructField("item_count", IntegerType(), True),
StructField("ingestion_timestamp", StringType(), True),
])

orders_df = spark.createDataFrame(orders_flat, schema=schema)

print(f"DataFrame shape: {orders_df.count()} rows, {len(orders_df.columns)} columns")
orders_df.printSchema()
orders_df.show(3, truncate=50)

```

`printSchema()` is your best friend when learning PySpark. It shows you the column names, data types, and nullability. Always run this after loading data. If a column that should be a `DoubleType` shows as `StringType`, you've found a data quality issue.

python

```

# =====
# CELL 4 - Write to Bronze Lakehouse in partitioned JSON
# =====

# IMPORTANT: In Fabric, your Lakehouse path uses the abfss:// scheme
# The path format is:
abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<lakehouse>.Lakehouse/Files/<folder>

# Fabric provides a shortcut: you can use the Lakehouse name directly
# First, let's write partitioned by date - critical for performance

# Add a partition column
orders_df = orders_df.withColumn(
    "partition_date",
    F.to_date(F.col("order_date")))
).withColumn(
    "year", F.year(F.col("order_date")))
).withColumn(
    "month", F.month(F.col("order_date")))
).withColumn(
    "day", F.dayofmonth(F.col("order_date")))

```

```

)

# Write to Bronze Lakehouse Files section
# In Fabric Notebooks attached to a Lakehouse, use 'Files/' prefix

bronze_path = "Files/raw/orders/"

orders_df.write \
    .mode("overwrite") \
    .partitionBy("year", "month", "day") \
    .json(bronze_path)

print("Orders written to Bronze Lakehouse!")
print(f"Path: {bronze_path}")
print(f"Partitioned by: year / month / day")

# Verify the write
verify_df = spark.read.json(bronze_path)
print(f"\nVerification read: {verify_df.count()} rows")

```

Why partition by year/month/day? Imagine your orders table has 3 years of data — that's ~10 million rows. If someone queries "show me all orders from January 15, 2024", without partitioning, Spark reads ALL 10 million rows to find the ones matching that date. With partitioning by year=2024/month=1/day=15, Spark only reads ~10,000 rows from that one folder. This is called **partition pruning** and it's the single biggest performance optimization in big data engineering.

python

```

# =====
# CELL 5 – Generate and save Customer data (CSV format)
# =====

import pandas as pd

def generate_customers(n=500):
    """Generate customer master data"""
    customers = []

    first_names = ["Rahul", "Priya", "Amit", "Sneha", "Vikram",
                  "Anita", "Suresh", "Kavya", "Rajan", "Meera",
                  "Arjun", "Divya", "Kiran", "Pooja", "Rohit"]
    last_names = ["Sharma", "Patel", "Singh", "Kumar", "Gupta",
                 "Joshi", "Mehta", "Reddy", "Nair", "Iyer"]

    domains = ["gmail.com", "yahoo.com", "outlook.com",
              "hotmail.com", "rediffmail.com"]

```

```

for i in range(n):
    first = random.choice(first_names)
    last = random.choice(last_names)
    join_date = datetime(2020, 1, 1) + timedelta(days=random.randint(0, 1460))

    customers.append({
        "customer_id": f"CUST{10000 + i}",
        "first_name": first,
        "last_name": last,
        # Intentional issue: some emails missing
        "email": f"{first.lower()}.{last.lower()}{i}@{random.choice(domains)}" \
            if random.random() > 0.03 else "",
        "phone": f"+91{random.randint(7000000000, 9999999999)}",
        "city": random_city_messy(), # messy again!
        "state": random.choice(["MH", "DL", "KA", "TS", "TN", "GJ", "RJ", "WB"]),
        "pincode": str(random.randint(100000, 999999)),
        "date_joined": join_date.strftime("%Y-%m-%d"),
        "loyalty_tier": random.choice(["Bronze", "Silver", "Gold", "Platinum"]),
        # Intentional issue: total_orders sometimes negative (data entry error)
        "total_orders": random.randint(-1, 150),
        "is_active": random.choice(["Y", "y", "Yes", "1", "TRUE"]) # messy boolean!
    })

```

```

return customers

```

```

customers = generate_customers(500)
print(f"Generated {len(customers)} customers")

```

```

# Convert to CSV string
output = io.StringIO()
writer = csv.DictWriter(output, fieldnames=customers[0].keys())
writer.writeheader()
writer.writerows(customers)
csv_content = output.getvalue()

```

```

# Save CSV to Bronze Lakehouse

```

```

import notebookutils
notebookutils.fs.put("Files/raw/customers/customers_20240101.csv", csv_content, overwrite=True)

```

```

print("Customers CSV saved to Bronze Lakehouse!")
print("First 3 rows:")
for c in customers[:3]:
    print(json.dumps(c, indent=2))

```

python

```

# =====
# CELL 6 – Generate and save Product data (simulated API response)
# =====

def generate_products(n=200):
    """Simulate a product catalog API response"""
    brands = ["Samsung", "Apple", "Nike", "Adidas", "Prestige",
              "Philips", "LG", "Boat", "Noise", "Wildcraft"]

    products = []
    for i in range(n):
        category = random.choice(CATEGORIES)
        base_price = round(random.uniform(199, 89999), 2)

        products.append({
            "product_id": f"PROD{1000 + i}",
            "product_name": f"{random.choice(brands)} {category} Model-
{random.randint(100,999)}",
            "category": category,
            "brand": random.choice(brands),
            "base_price": base_price,
            # Intentional: selling price sometimes higher than base (data error)
            "selling_price": round(base_price * random.uniform(0.7, 1.05), 2),
            "stock_quantity": random.randint(0, 500),
            "supplier_id": f"SUP{random.randint(100, 199)}",
            "is_active": random.choice([True, False]) if random.random() > 0.9 else True,
            "created_at": (datetime(2022, 1, 1) + timedelta(days=random.randint(0,
700))).isoformat(),
            "updated_at": datetime.now().isoformat(),
            # Intentional: some products missing weight (incomplete data)
            "weight_kg": round(random.uniform(0.1, 25.0), 2) if random.random() > 0.15 else
None,
            "rating": round(random.uniform(1.0, 5.0), 1),
            "review_count": random.randint(0, 5000),
        })

    return products

products = generate_products(200)

# Save as JSON (simulating API response format)
products_json = json.dumps({"data": products, "total": len(products),
                           "fetched_at": datetime.now().isoformat()}, indent=2)

notebookutils.fs.put("Files/raw/products/products_20240101.json", products_json,
overwrite=True)
print(f"Saved {len(products)} products as API-style JSON")

```



```
print("Done! All synthetic data generated and saved to Bronze Lakehouse.")
```

Run all cells now. After they complete, go to your `bronze_lakehouse` and click the Files section. You'll see:

```
Files/  
  raw/  
    orders/    ← partitioned JSON files (year=2024/month=1/day=1/ etc.)  
    customers/ ← customers_20240101.csv  
    products/  ← products_20240101.json
```

This is your Bronze layer — raw, unmodified source data exactly as it arrived.

2.4 Ingestion Method 2 — Build a Real Data Pipeline

Now let's build a proper pipeline that automates loading new data files from a "landing zone" into the Bronze Lakehouse. This is the pattern companies use for vendor file drops, SFTP deliveries, and scheduled batch loads.

Step 1 — Create the Pipeline

1. In your workspace, click **+ New item** → **Data pipeline**
2. Name: `pl_ingest_landing_to_bronze`
3. Click **Create**

Step 2 — Add a Copy Data Activity for Orders

1. From the top toolbar, click **Copy data**
2. A "Copy data1" box appears. Click it to select it.
3. Rename it: click the pencil icon on the activity → type `copy_orders_to_bronze`

Configure Source tab:

1. Click **Source** tab in the bottom panel
2. Click **+ New** next to Connection
3. Search for **Azure Data Lake Storage Gen2**

Wait — why ADLS Gen2? Because OneLake (Fabric's storage) IS ADLS Gen2 under the hood. The Lakehouse Files section is an ADLS Gen2 path. This is the connector we use to read from one Lakehouse location and write to another within the same OneLake.

4. Select **Azure Data Lake Storage Gen2** → Continue
5. For **Account name/URL**, use your OneLake path: `https://onelake.dfs.fabric.microsoft.com`
6. Authentication: **Organizational account** → Sign in with your Fabric account

7. Name the linked service: `ls_onelake`
8. Test connection → should show "Connection successful"
9. Click **Create**

Set Source path:

- File path container: your workspace name
- File path directory: `bronze_lakehouse.Lakehouse/Files/raw/orders`
- File format: **JSON** → recursive: **True**

Configure Destination (Sink) tab:

1. Click **Destination** tab
2. Connection: same `ls_onelake` linked service
3. File path: `bronze_lakehouse.Lakehouse/Files/processed/orders`
4. File format: **Parquet** (we convert from JSON to Parquet for better compression)
5. Copy behavior: **Preserve hierarchy**

Why convert JSON to Parquet in the pipeline? Parquet is a columnar format — it's 5–10x smaller than JSON and 10–100x faster to query because analytics engines only read the columns they need. Many teams land as JSON (source format preservation) and convert to Parquet in the same pipeline step. This is valid in Bronze — you preserve the original values but optimize the storage format.

Step 3 — Add Pipeline Parameters (Production Pattern)

Hard-coding paths in pipelines is an anti-pattern. In production, you parameterize pipelines so the same pipeline can handle any date or any data type.

1. Click on blank canvas (not on any activity)
2. At the bottom, click **Parameters** tab
3. Click **+ New**:
 - Name: `p_load_date`
 - Type: `String`
 - Default value: `@formatDateTime(utcNow(), 'yyyy-MM-dd')`

The `@formatDateTime(utcNow(), 'yyyy-MM-dd')` expression uses Fabric's expression language to automatically use today's date as the default. This means when you run the pipeline manually without specifying a date, it uses today. When you schedule it, it still uses the execution date. This is the pattern for daily incremental loads.

Step 4 — Add Error Handling with a Validation Activity

In production, you never blindly copy data — you validate first.

1. From the toolbar, add a **Validation** activity
2. Name it: `validate_source_files_exist`
3. Drag it BEFORE the copy activity
4. Connect them: hover over the Validation box → drag the green arrow to the Copy activity

Configure Validation:

- Dataset: point to your source JSON files
- Timeout: `00:01:00` (fail after 1 minute if files not found)
- Sleep: `10` (check every 10 seconds)
- Minimum size: `1` (fail if file is 0 bytes — a common real issue)

Why validate first? Companies have wasted entire pipeline runs because a source system delivered an empty file due to a bug on their end. The pipeline happily "succeeded" by copying nothing, and the team only discovered it 24 hours later when reports showed no data. A validation activity catches this in 60 seconds.

Step 5 — Add Notifications (Production Must-Have)

1. Click on the Pipeline canvas background
2. In top menu, click **Activities** → **Fail**
3. Name it: `notify_on_failure`
4. Connect: from Copy activity's red (failure) output → to this Fail activity

In the Fail activity settings:

- Message:
`@concat('Pipeline failed for load date: ', pipeline().parameters.p_load_date, ' — Error: ', activity('copy_orders_to_bronze').error.message)`
- Error code: `INGEST_FAILURE_001`

The `@concat()` and `activity().error.message` are pipeline expressions. They dynamically build the error message at runtime. Every message should include: which pipeline, which date, and the actual error. In production, this message goes to Teams or email via an Alert rule you set in Fabric Monitor.

Step 6 — Run and Monitor Your Pipeline

1. Click **Run** (the play button) in the top toolbar

2. A dialog appears — leave `p_load_date` as default

3. Click **OK**

You'll see the pipeline execute in real time:

- Each activity box shows a spinning icon while running
- Green checkmark = success
- Red X = failure

Click on any activity while it's running or after it completes → **Details** → you'll see:

- Rows read / rows written
- Data volume transferred
- Duration
- Any error messages

This monitoring view is what you show in daily standups: "Pipeline ran at 2am, copied 9,847 rows, took 4 minutes, no errors." This is production data engineering.

2.5 Ingestion Method 3 — Dataflow Gen2 (For Customer CSV)

Dataflow Gen2 is Power Query in the cloud. If you've ever used Excel's "Get Data" or Power BI's Power Query editor, this is the same engine — but it runs on Spark and can handle millions of rows.

When to use Dataflow Gen2 vs Notebooks?

Use Dataflow Gen2 when your transformations can be done with clicks rather than code — filtering columns, renaming headers, changing data types, merging tables. It's perfect for analysts who aren't comfortable with PySpark. Use Notebooks when you need loops, custom functions, ML models, or complex logic.

Step-by-Step: Create a Dataflow Gen2 for Customer Data

1. In your workspace, click **+ New item**
2. Select **Dataflow Gen2**
3. Name: `df_ingest_customers_bronze`

You're now in the Power Query editor.

Step 1 — Connect to the Source CSV

1. Click **Get data** (top-left)
2. Search for **Lakehouse** and select it
3. In the browser that opens, navigate to `bronze_lakehouse` → `Files` → `raw` → `customers`

4. Select `customers_20240101.csv`

5. Click **Create**

You'll see a preview of your data. Note the issues immediately visible:

- `is_active` column has values like "Y", "y", "Yes", "1", "TRUE" (the intentional mess we created)
- `total_orders` has negative values
- `city` has messy spellings

Step 2 — Apply Transformations with Clicks

Fix column data types:

1. Click the column header for `date_joined`
2. Click the **ABC** icon → select **Date**
3. Dataflow automatically generates the M code:
`Table.TransformColumnTypes(Source, {"date_joined", type date})`

Filter out invalid records:

1. Click the dropdown arrow on `total_orders`
2. Select **Number filters** → **Greater than or equal to** → enter `0`
3. This removes the negative-order-count rows

Add a load timestamp column:

1. In the ribbon, click **Add column** → **Custom column**
2. Name: `ingestion_timestamp`
3. Formula: `DateTime.LocalNow()`

Step 3 — Configure the Output Destination

This is the key Dataflow Gen2 feature — you define WHERE the output goes.

1. In the bottom-right panel, click **+ Add data destination**
2. Select **Lakehouse**
3. Navigate to `bronze_lakehouse`
4. In the "Tables" section, type: `raw_customers` (this creates a Delta table)
5. Update method: **Replace** (overwrite on each run)
6. Click **Save settings**

Notice we're writing to **Tables** (Delta format) not Files. This is a common pattern for Dataflow Gen2 — it creates managed Delta tables. For true Bronze (raw preservation), you might prefer

Files, but for customers (a relatively stable dataset), writing directly to a Delta table in Bronze is acceptable.

Step 4 – Publish the Dataflow

1. Click **Publish** (top-right)
2. The Dataflow saves and runs

After it completes, go to `bronze_lakehouse` → **Tables** section. You'll see `raw_customers` as a Delta table. Click it to preview the data.

2.6 Incremental Loading – The Most Important Production Pattern

What is incremental loading?

Full load = copy ALL data every time. If your orders table has 50 million rows and you run a full load daily, you copy 50 million rows every single day. At 30 days, that's 1.5 billion row-copies that could have been 30 million (just the new ones each day).

Incremental load = copy only NEW or CHANGED data since the last run.

The Watermark Pattern

The industry standard for incremental loading is the **watermark pattern**:

1. Store the "high watermark" — the maximum value of your load key (usually a date or auto-increment ID) from the last successful run
2. On next run, only pull records WHERE `updated_at > last_watermark`
3. After successful load, update the watermark to the new maximum

Let's implement this properly in a Notebook:

```
python

# =====
# NOTEBOOK: nb_incremental_orders_load
# =====
# This notebook implements the watermark pattern for incremental
# ingestion from a source system into Bronze Lakehouse

from pyspark.sql import functions as F
from pyspark.sql.types import import *
from delta.tables import DeltaTable
import json
from datetime import datetime

# =====
```

```

# STEP 1: Read or initialize the watermark
# -----
# The watermark table tracks the last successful load timestamp
# per pipeline (allows multiple pipelines to share this pattern)

WATERMARK_TABLE = "bronze_lakehouse.watermark_control"
PIPELINE_NAME = "orders_incremental_load"

def get_watermark(pipeline_name: str) -> str:
    """
    Retrieves the last successful load watermark.
    Returns '1900-01-01' for first run (pulls all data).
    """
    try:
        wm_df = spark.sql(f"""
            SELECT max_watermark
            FROM {WATERMARK_TABLE}
            WHERE pipeline_name = '{pipeline_name}'
            """)

        if wm_df.count() == 0:
            print(f"No watermark found for {pipeline_name}. First run - full load.")
            return "1900-01-01T00:00:00"

        watermark = wm_df.collect()[0][0]
        print(f"Last watermark: {watermark}")
        return watermark

    except Exception as e:
        print(f"Watermark table not found. Creating it. Error: {e}")
        # Create the watermark control table if it doesn't exist
        spark.sql(f"""
            CREATE TABLE IF NOT EXISTS {WATERMARK_TABLE} (
                pipeline_name STRING,
                max_watermark STRING,
                last_run_timestamp TIMESTAMP,
                rows_loaded LONG
            )
            USING DELTA
            """)
        return "1900-01-01T00:00:00"

# Get current watermark
last_watermark = get_watermark(PIPELINE_NAME)
print(f>Loading orders modified after: {last_watermark}")

```

python

```

# -----
# STEP 2: Read only new/changed records
# -----
# In production, this would be a JDBC query to your source DB:
# df = spark.read.jdbc(url, "orders",
#     properties={"query": f"SELECT * FROM orders WHERE updated_at > '{last_watermark}'"})
#
# In our case, we read from the Bronze Files and filter by ingestion_timestamp

source_df = spark.read.json("Files/raw/orders/")

# Filter: only records after the watermark
incremental_df = source_df.filter(
    F.col("ingestion_timestamp") > last_watermark
)

new_record_count = incremental_df.count()
print(f"New records to load: {new_record_count:,}")

if new_record_count == 0:
    print("No new records. Exiting gracefully.")
    dbutils.notebook.exit("NO_NEW_DATA")

```

python

```

# -----
# STEP 3: Write incremental data using MERGE (Upsert)
# -----
# MERGE is the Delta Lake superpower:
# - If order_id already exists → UPDATE it (handles re-delivered records)
# - If order_id is new → INSERT it
# This prevents duplicates even if source sends records twice

TARGET_TABLE = "bronze_lakehouse.raw_orders_incremental"

# Check if target Delta table exists
target_exists = spark.catalog.tableExists(TARGET_TABLE)

if not target_exists:
    print(f"Creating target table: {TARGET_TABLE}")
    incremental_df.write \
        .format("delta") \
        .mode("overwrite") \
        .option("overwriteSchema", "true") \
        .saveAsTable(TARGET_TABLE)
    print(f"Table created with {new_record_count:,} rows")

```



```

else:
    print(f"Merging {new_record_count:,} new records into {TARGET_TABLE}")

    target_table = DeltaTable.forName(spark, TARGET_TABLE)

    target_table.alias("target").merge(
        incremental_df.alias("source"),
        "target.order_id = source.order_id"    # Match on business key
    ).whenMatchedUpdateAll(                    # If exists: update ALL columns
    ).whenNotMatchedInsertAll(                 # If new: insert all columns
    ).execute()

    print("Merge complete!")

# Verify
final_count = spark.sql(f"SELECT COUNT(*) FROM {TARGET_TABLE}").collect()[0][0]
print(f"Total rows in target: {final_count:,}")

```

python

```

# -----
# STEP 4: Update the watermark
# -----
# Only update watermark AFTER successful write
# If the write fails, we don't update - next run will retry

new_watermark = incremental_df.agg(
    F.max("ingestion_timestamp")
).collect()[0][0]

print(f"Updating watermark to: {new_watermark}")

spark.sql(f"""
MERGE INTO {WATERMARK_TABLE} AS target
USING (SELECT
    '{PIPELINE_NAME}' AS pipeline_name,
    '{new_watermark}' AS max_watermark,
    current_timestamp() AS last_run_timestamp,
    {new_record_count} AS rows_loaded
) AS source
ON target.pipeline_name = source.pipeline_name
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
""")

print(f"""
=== LOAD SUMMARY ===

```

```

Pipeline:      {PIPELINE_NAME}
Previous mark: {last_watermark}
New mark:      {new_watermark}
Records loaded: {new_record_count:,}
Run time:      {datetime.now().isoformat()}
=====
"""

```

The watermark pattern is tested in almost every senior data engineer interview. Know it cold: what it is, why you need it, what happens if the pipeline fails mid-run (you don't update the watermark, so the next run retries the same window — this is called "at-least-once" delivery), and what MERGE does differently from INSERT.

2.7 Data Ingestion — Common Mistakes and Best Practices

Ingestion: wrong approach vs production approach

Common mistake	Production approach
Full load every day (copies all data) Kills source DB, wastes storage	Watermark-based incremental load Only new/changed rows per run
Hard-code source paths in pipeline Breaks when folder structure changes	Pipeline parameters + linked services One change updates all pipelines
Transform in Bronze (modify raw data) Lose original record forever	Bronze = raw, never modified Transform in Silver Notebook
No partitioning on large tables Full table scan on every query	Partition by date (year/month/day) Queries scan only needed partitions
No error handling in pipeline Silent failures corrupt downstream	Validate → Copy → Update watermark Fail fast, alert, retry safely
INSERT on every run (duplicates!) Same order appears 30 times	MERGE (upsert) by business key Idempotent: safe to re-run anytime

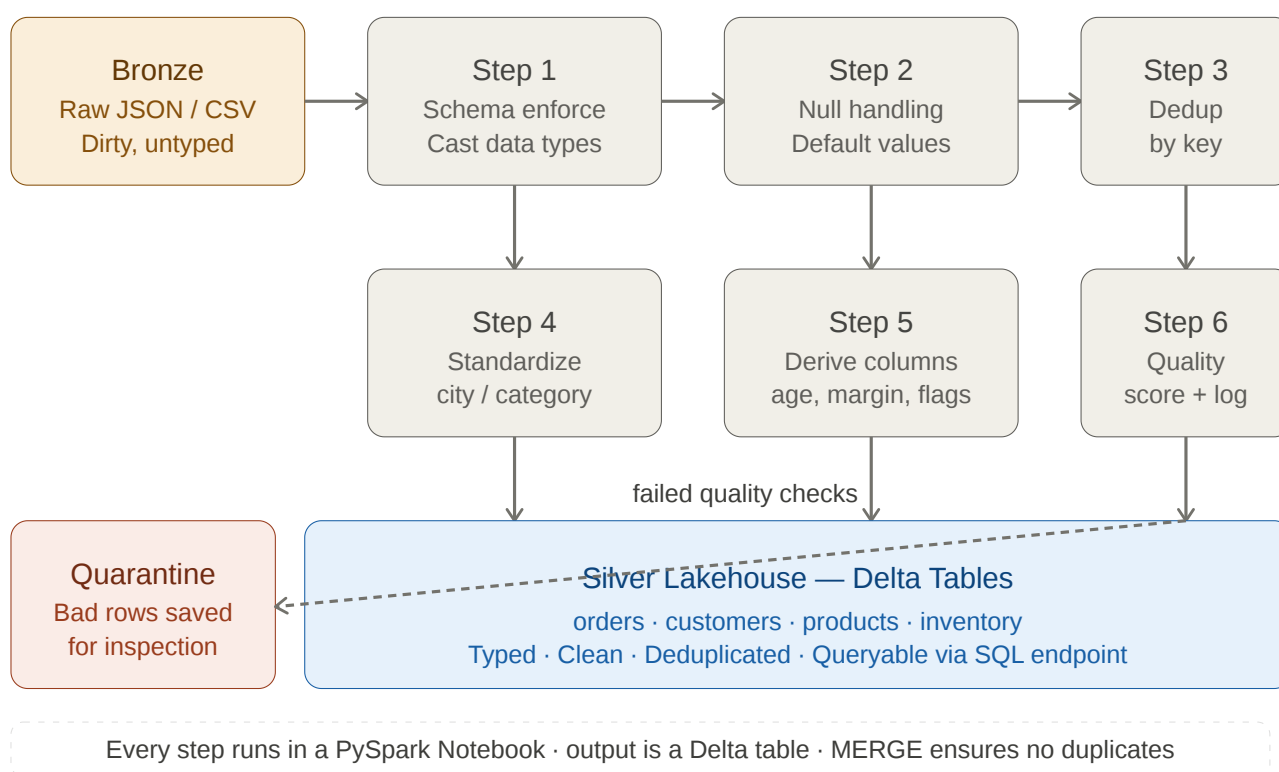




Phase 3 — Data Transformation (Silver Layer)

This is where you become a real data engineer. Ingestion is about moving data. Transformation is about making it trustworthy, consistent, and usable. Every company's competitive advantage sits in how well they clean and model their data.

Bronze → Silver: what actually happens inside the Notebook



3.1 Why Transformation Is the Heart of Data Engineering

What is it?

Transformation is the process of converting raw, messy Bronze data into clean, trusted, business-ready Silver data. It is not just "cleaning" — it is applying **business rules**, **domain knowledge**, and **quality standards** so that every analyst who touches the Silver layer sees the same, consistent truth.

Why does it matter so deeply?

Here is a real scenario that happens at companies every week. The sales team reports revenue of ₹12.4 crore. The finance team reports ₹11.9 crore. Both are querying the same underlying data but through different paths — and neither has applied consistent rules for what counts as "revenue" (is a

cancelled order revenue? What about a return that hasn't been processed yet?). The CEO asks which number to trust. Neither team knows. This is the **data trust crisis**, and transformation is the cure.

When your Silver layer is correct, **there is only one number**. Every team querying Silver gets the same answer because the rules were applied once, at the transformation stage, by the data engineering team.

What problems does it solve for RetailPulse?

Raw data problem	Silver transformation fix
city = "mmbai", "MUMBAI", "MUM"	Standardized to "Mumbai" using a reference map
amount = "1299.00" (string)	Cast to DECIMAL(12,2)
total_orders = -3 (impossible)	Flagged as bad, sent to quarantine table
customer_id = null (5% of orders)	Flagged with is_guest_order = true
Same order delivered twice from source	MERGE deduplicates by order_id
is_active = "Y", "y", "Yes", "1"	Standardized to boolean true
Selling price > base price (illogical)	Flagged as price_anomaly = true

3.2 Setting Up the Silver Transformation Notebook

Step 1 – Create the Notebook and Attach It to Your Lakehouses

In your Fabric workspace:

- 1. Click **+ New item** → **Notebook**
- 2. Name: nb_bronze_to_silver_orders
- 3. Click **Create**

Now attach your Lakehouses so the notebook can read Bronze and write Silver:

- 1. In the left Explorer panel of the Notebook, click **Add Lakehouse**
- 2. Add bronze_lakehouse – this becomes your default lakehouse (read source)
- 3. Click **Add Lakehouse** again → add silver_lakehouse

When you add a Lakehouse to a Notebook, Fabric mounts it at a specific path. The first Lakehouse you add becomes the "default" – when you write `spark.read.json("Files/raw/orders/")` , it reads from the default Lakehouse. To read from or write to a non-default Lakehouse, use the full `abfss://` path or the Lakehouse name prefix.

Step 2 – The Complete Bronze → Silver Orders Transformation

Run each cell below in sequence. Every line is explained.

python

```
# =====
# CELL 1 – Imports and configuration
# =====

from pyspark.sql import functions as F
from pyspark.sql import types as T
from pyspark.sql.window import Window
from delta.tables import DeltaTable
from datetime import datetime

# — Configuration dictionary —
# Professional engineers never scatter magic strings through code.
# Put all configuration in one place at the top.
config = {
    "bronze_orders_path": "Files/raw/orders/",
    "silver_orders_table": "silver_lakehouse.silver_orders",
    "quarantine_table": "silver_lakehouse.quarantine_orders",
    "watermark_table": "silver_lakehouse.etl_watermark",
    "pipeline_name": "bronze_to_silver_orders",
    "dq_pass_threshold": 0.95, # 95% of rows must pass quality checks
}

print("Configuration loaded.")
print(f"Source: {config['bronze_orders_path']}")
print(f"Target: {config['silver_orders_table']}")
print(f"Run time: {datetime.now().isoformat()}")
```

python

```
# =====
# CELL 2 – Read from Bronze with schema enforcement
# =====
# Reading raw JSON without a schema lets Spark infer types.
# That's dangerous: today "amount" is inferred as Double.
# Tomorrow a vendor sends "1,299.00" and it's inferred as String.
# Your pipeline silently breaks. Always define your schema explicitly.

raw_schema = T.StructType([
    T.StructField("order_id", T.StringType(), False),
    T.StructField("order_date", T.StringType(), True),
    T.StructField("customer_id", T.StringType(), True), # nullable: guest orders
    T.StructField("city", T.StringType(), True),
```

```

    T.StructField("state",          T.StringType(),    True),
    T.StructField("payment_method", T.StringType(),    True),
    T.StructField("order_status",   T.StringType(),    True),
    T.StructField("order_total",    T.DoubleType(),    True),
    T.StructField("discount_code",  T.StringType(),    True), # nullable: no promo
    T.StructField("items_json",     T.StringType(),    True),
    T.StructField("item_count",     T.IntegerType(),   True),
    T.StructField("ingestion_timestamp", T.StringType(),    True),
    T.StructField("year",           T.IntegerType(),   True),
    T.StructField("month",          T.IntegerType(),   True),
    T.StructField("day",            T.IntegerType(),   True),
])

# mode="PERMISSIVE" = bad rows get null fields + written to _corrupt_record
# FAILFAST would abort on first bad row (use in strict production pipelines)
# DROPMALFORMED silently drops bad rows (dangerous - you lose data silently)
raw_df = spark.read \
    .schema(raw_schema) \
    .option("mode", "PERMISSIVE") \
    .option("columnNameOfCorruptRecord", "_corrupt_record") \
    .json(config["bronze_orders_path"])

total_raw = raw_df.count()
print(f"Raw rows read from Bronze: {total_raw:,}")
raw_df.printSchema()
raw_df.show(3, truncate=60)

```

python

```

# =====
# CELL 3 - Step 1: Fix data types and parse timestamps
# =====
# This is the most critical step. Every column gets an explicit,
# correct data type. Wrong types cause silent wrong calculations
# (summing a string column returns null, not an error).

typed_df = raw_df \
    .withColumn(
        # Convert ISO string to proper timestamp
        # F.to_timestamp understands ISO 8601 automatically
        "order_timestamp",
        F.to_timestamp(F.col("order_date"), "yyyy-MM-dd'T'HH:mm:ss")
    ) \
    .withColumn(
        # Extract just the date for partitioning and daily aggregations
        "order_date_key",
        F.to_date(F.col("order_date"), "yyyy-MM-dd'T'HH:mm:ss")
    )

```

```

) \
.withColumn(
    # order_total: handle both numeric and string representations
    # If it's already a Double, cast is a no-op.
    # If source sends "1299.00", this converts it correctly.
    # If source sends "1,299.00" (comma), this returns null – caught later.
    "order_amount",
    F.col("order_total").cast(T.DecimalType(12, 2))
) \
.withColumn(
    # Extract hour of day for time-based analytics
    # "Did orders peak at lunch or evening?"
    "order_hour",
    F.hour(F.col("order_timestamp"))
) \
.withColumn(
    # Day of week: 1=Sunday ... 7=Saturday in Spark
    # Useful for "weekend vs weekday" analysis
    "day_of_week",
    F.dayofweek(F.col("order_timestamp"))
) \
.withColumn(
    # Boolean flag: is this a weekday order?
    "is_weekend",
    F.dayofweek(F.col("order_timestamp")).isin([1, 7])
)

print(f"After type casting: {typed_df.count():,} rows")
# Show null counts per column – your first data quality signal
null_counts = typed_df.select([
    F.count(F.when(F.col(c).isNull(), c)).alias(c)
    for c in typed_df.columns
])
print("\nNull counts per column:")
null_counts.show(vertical=True)

```

python

```

# =====
# CELL 4 – Step 2: Standardize city names (business rules)
# =====
# This is where BUSINESS KNOWLEDGE meets DATA ENGINEERING.
# You cannot write this code without talking to the business.
# "Which spelling is canonical?" is a business decision, not technical.

# Build a lookup dictionary for city standardization
# In production, this would come from a reference/master data table

```

```

# managed by the business – not hardcoded here.
city_mapping = {
    # Mumbai variants
    "mumbai": "Mumbai", "mum": "Mumbai", "mmbai": "Mumbai",
    "bombay": "Mumbai", "bom": "Mumbai",
    # Delhi variants
    "delhi": "Delhi", "new delhi": "Delhi", "ncr": "Delhi",
    "new_delhi": "Delhi", "dl": "Delhi",
    # Bangalore variants
    "bangalore": "Bangalore", "bengaluru": "Bangalore",
    "blr": "Bangalore", "bengalore": "Bangalore",
    # Hyderabad variants
    "hyderabad": "Hyderabad", "hyd": "Hyderabad",
    "secunderabad": "Hyderabad",
    # Chennai variants
    "chennai": "Chennai", "madras": "Chennai", "chn": "Chennai",
    # Other cities
    "pune": "Pune", "kolkata": "Kolkata", "calcutta": "Kolkata",
    "ahmedabad": "Ahmedabad", "amdavad": "Ahmedabad",
    "jaipur": "Jaipur", "surat": "Surat",
}

# Register as Spark UDF (User Defined Function)
# A UDF is a Python function you apply to each row in a DataFrame
# Use UDFs when no built-in Spark function exists for your logic
def standardize_city(city_raw: str) -> str:
    if city_raw is None:
        return "Unknown"

    # Normalize: lowercase, strip spaces
    normalized = city_raw.lower().strip()
    # Look up in our mapping
    return city_mapping.get(normalized, city_raw.title())
    # .title() capitalizes first letter of each word as fallback
    # So "mysore" → "Mysore" even if not in our mapping

# Register with Spark. The return type must be declared.
standardize_city_udf = F.udf(standardize_city, T.StringType())

# Also standardize order_status to a controlled vocabulary
valid_statuses = {"placed", "confirmed", "shipped", "delivered",
                  "cancelled", "returned"}

def standardize_status(status: str) -> str:
    if status is None:
        return "unknown"
    normalized = status.lower().strip()
    return normalized if normalized in valid_statuses else "unknown"

```



```

standardize_status_udf = F.udf(standardize_status, T.StringType())

# Apply standardizations
standardized_df = typed_df \
    .withColumn("city_clean", standardize_city_udf(F.col("city"))) \
    .withColumn("status_clean", standardize_status_udf(F.col("order_status"))) \
    .withColumn(
        # Uppercase state codes for consistency
        "state_clean",
        F.upper(F.trim(F.col("state")))
    ) \
    .withColumn(
        # Standardize payment method casing
        "payment_method_clean",
        F.initcap(F.trim(F.col("payment_method")))
    )

print("City standardization applied.")

# QA check: see what cities we standardized
standardized_df \
    .groupBy("city", "city_clean") \
    .count() \
    .orderBy(F.desc("count")) \
    .show(20, truncate=False)

```

python

```

# =====
# CELL 5 – Step 3: Handle nulls with business rules
# =====
# There are three ways to handle nulls. The right one depends
# on what the null MEANS in your business context.
#
# Strategy A – Default value: null discount_code means no promo used
# Strategy B – Flag the row: null customer_id means guest checkout
# Strategy C – Reject the row: null order_id means data is corrupt

null_handled_df = standardized_df \
    .withColumn(
        # Strategy A: nulls here are business-meaningful (no promo code used)
        "discount_code_clean",
        F.coalesce(F.col("discount_code"), F.lit("NO_PROMO"))
        # F.coalesce returns the first non-null value from its arguments
        # If discount_code is null, returns "NO_PROMO"
    ) \
    .withColumn(

```

```

    # Strategy B: flag null customer as guest order (business concept)
    "is_guest_order",
    F.col("customer_id").isNull()
) \
.withColumn(
    # Replace null customer_id with a generated guest ID
    # Business needs something to group by even for guests
    "customer_id_clean",
    F.when(
        F.col("customer_id").isNull(),
        F.concat(F.lit("GUEST_"), F.col("order_id"))
    ).otherwise(F.col("customer_id"))
) \
.withColumn(
    # Null amount is a hard error – flag it for quarantine
    "has_amount_error",
    F.col("order_amount").isNull()
) \
.withColumn(
    # Null timestamp is a hard error
    "has_date_error",
    F.col("order_timestamp").isNull()
)

# F.when() is PySpark's version of IF-THEN-ELSE
# F.when(condition, value_if_true).otherwise(value_if_false)
# You can chain: F.when(cond1, v1).when(cond2, v2).otherwise(vN)

print("Null handling applied.")

```

python

```

# =====
# CELL 6 – Step 4: Derive business columns
# =====
# Derived columns encode business logic once, centrally.
# Instead of every analyst writing their own version of
# "is this a high-value order?", we define it here.
# This is the data engineering team adding business value.

enriched_df = null_handled_df \
    .withColumn(
        # Order size tier – business definition of "small/medium/large"
        # Talk to the business before defining these thresholds!
        "order_size_tier",
        F.when(F.col("order_amount") < 500, F.lit("Small"))
        .when(F.col("order_amount") < 2000, F.lit("Medium"))
    )

```

```

        .when(F.col("order_amount") < 10000, F.lit("Large"))
        .otherwise(F.lit("Premium"))
    ) \
    .withColumn(
        # Days since order – useful for aging analysis
        # F.datediff(end, start) returns integer days between dates
        "days_since_order",
        F.datediff(F.current_date(), F.col("order_date_key"))
    ) \
    .withColumn(
        # Is this order eligible for return? (within 30 days, not cancelled)
        "is_return_eligible",
        (F.col("days_since_order") <= 30) &
        (F.col("status_clean").isin(["delivered"]))
    ) \
    .withColumn(
        # Has a discount been applied?
        "has_discount",
        F.col("discount_code_clean") != "NO_PROMO"
    ) \
    .withColumn(
        # Record insertion timestamp for lineage tracking
        # This tells you WHEN this Silver record was created
        "silver_insert_timestamp",
        F.current_timestamp()
    ) \
    .withColumn(
        # Source layer for data lineage
        "source_layer",
        F.lit("bronze_orders")
    )
)

print(f"Enrichment applied: {enriched_df.count():,} rows")

```

python

```

# =====
# CELL 7 – Step 5: Deduplication using Window functions
# =====
# The most sophisticated step. In real data, the same order_id
# can appear multiple times because:
# - The source system resent a batch
# - A pipeline ran twice (accidental double load)
# - An order was updated (status change) creating a second record
#
# We keep only the LATEST version of each order_id.

```

```

# Window function: for each order_id, rank rows by ingestion time
# Row 1 = most recent record for that order_id
dedup_window = Window \
    .partitionBy("order_id") \
    .orderBy(F.desc("ingestion_timestamp"))
# partitionBy = "group by" for window functions
# orderBy desc = most recent first

dedup_df = enriched_df \
    .withColumn("row_num", F.row_number().over(dedup_window)) \
    .filter(F.col("row_num") == 1) \
    .drop("row_num")
# row_number() assigns 1,2,3... within each partition
# We keep only row_num == 1 (the most recent per order_id)

duplicates_removed = enriched_df.count() - dedup_df.count()
print(f"Rows before dedup: {enriched_df.count():,}")
print(f"Rows after dedup: {dedup_df.count():,}")
print(f"Duplicates removed: {duplicates_removed:,}")

```

python

```

# =====
# CELL 8 – Step 6: Data Quality scoring and quarantine
# =====
# Every row gets a quality score. Rows below the threshold
# go to a QUARANTINE table, not the Silver table.
# This is enterprise-grade DQ – not just logging errors,
# but ROUTING them so bad data never touches downstream models.

dq_df = dedup_df.withColumn(
    "dq_score",
    # Each check is worth equal weight (0.2 each, total = 1.0)
    # Add more checks to make scoring more granular
    (
        # Check 1: order_id is never null
        F.when(F.col("order_id").isNotNull(), 0.2).otherwise(0.0) +
        # Check 2: order amount is positive
        F.when(F.col("order_amount") > 0, 0.2).otherwise(0.0) +
        # Check 3: order timestamp is not null
        F.when(F.col("order_timestamp").isNotNull(), 0.2).otherwise(0.0) +
        # Check 4: status is in controlled vocabulary
        F.when(F.col("status_clean") != "unknown", 0.2).otherwise(0.0) +
        # Check 5: city was successfully standardized (not "Unknown")
        F.when(F.col("city_clean") != "Unknown", 0.2).otherwise(0.0)
    )
).withColumn(

```

```

    "dq_passed",
    F.col("dq_score") >= 0.6    # Pass if at least 3/5 checks pass
)

# Split into GOOD and QUARANTINE DataFrames
good_df      = dq_df.filter(F.col("dq_passed") == True)
quarantine_df = dq_df.filter(F.col("dq_passed") == False)

good_count = good_df.count()
bad_count  = quarantine_df.count()
pass_rate  = good_count / dq_df.count()

print(f"Quality check results:")
print(f"  Passed:  {good_count:,} rows ({pass_rate:.1%})")
print(f"  Failed:  {bad_count:,} rows ({1-pass_rate:.1%})")
print(f"  Threshold: {config['dq_pass_threshold']:.0%}")

# Pipeline quality gate – if pass rate is too low, ABORT
# Don't write garbage Silver data just to say the pipeline "ran"
if pass_rate < config["dq_pass_threshold"]:
    raise Exception(
        f"DATA QUALITY GATE FAILED: only {pass_rate:.1%} of rows passed "
        f"(threshold: {config['dq_pass_threshold']:.0%}). "
        f"Check quarantine table for details."
    )
print("Quality gate passed!")

```

python

```

# =====
# CELL 9 – Step 7: Select final Silver columns and write
# =====
# Don't carry ALL Bronze columns into Silver.
# Be intentional about what Silver contains.
# Silver is curated, not a dump.

silver_df = good_df.select(
    # — Identity columns

    F.col("order_id"),
    F.col("customer_id_clean").alias("customer_id"),
    F.col("is_guest_order"),

    # — Time columns

    F.col("order_timestamp"),
    F.col("order_date_key").alias("order_date"),

```

```

F.col("order_hour"),
F.col("day_of_week"),
F.col("is_weekend"),
F.col("days_since_order"),

# — Geography columns

F.col("city_clean").alias("city"),
F.col("state_clean").alias("state"),

# — Financial columns

F.col("order_amount"),
F.col("order_size_tier"),
F.col("has_discount"),
F.col("discount_code_clean").alias("discount_code"),

# — Status and lifecycle

F.col("status_clean").alias("order_status"),
F.col("payment_method_clean").alias("payment_method"),
F.col("is_return_eligible"),
F.col("item_count"),
F.col("items_json"),

# — Data quality and lineage

F.col("dq_score"),
F.col("silver_insert_timestamp"),
F.col("source_layer"),
)

# Write to Silver using MERGE (upsert)
# Prevents duplicates even if this notebook runs twice
TARGET = config["silver_orders_table"]
target_exists = spark.catalog.tableExists(TARGET)

if not target_exists:
    print(f"Creating Silver table: {TARGET}")
    silver_df.write \
        .format("delta") \
        .mode("overwrite") \
        .partitionBy("order_date") \
        .saveAsTable(TARGET)
else:
    print(f"Merging into Silver table: {TARGET}")
    target_table = DeltaTable.forName(spark, TARGET)

```

```
target_table.alias("target").merge(
    silver_df.alias("source"),
    "target.order_id = source.order_id"
) \
.whenMatchedUpdateAll() \
.whenNotMatchedInsertAll() \
.execute()
```

```
final_count = spark.sql(f"SELECT COUNT(*) FROM {TARGET}").collect()[0][0]
print(f"Silver orders table: {final_count:,} rows total")
```

python

```
# =====
# CELL 10 - Write quarantine records with failure reason
# =====

if bad_count > 0:
    quarantine_final = quarantine_df.select(
        F.col("order_id"),
        F.col("order_date"),
        F.col("dq_score"),
        F.col("has_amount_error"),
        F.col("has_date_error"),
        F.col("order_amount"),
        F.col("order_status"),
        F.col("city"),
        F.current_timestamp().alias("quarantine_timestamp"),
        F.lit(config["pipeline_name"]).alias("failed_pipeline"),
    ).withColumn(
        "failure_reason",
        F.when(F.col("has_amount_error"), F.lit("NULL_AMOUNT"))
        .when(F.col("has_date_error"), F.lit("NULL_TIMESTAMP"))
        .otherwise(F.lit("DQ_SCORE_BELOW_THRESHOLD"))
    )

    quarantine_final.write \
        .format("delta") \
        .mode("append") \
        .saveAsTable(config["quarantine_table"])

    print(f"Quarantine: {bad_count:,} bad rows written for investigation")
else:
    print("No quarantine records - all rows passed quality checks!")
```

python

```

# =====
# CELL 11 – Validate Silver with SQL queries
# =====
# Always validate your output. Look at the data, don't just
# trust that "no error = correct data".

print("=== Silver Layer Validation ===\n")

# 1. Row count and date range
spark.sql(f"""
    SELECT
        COUNT(*)                AS total_orders,
        MIN(order_date)         AS earliest_order,
        MAX(order_date)         AS latest_order,
        COUNT(DISTINCT customer_id) AS unique_customers,
        COUNT(DISTINCT city)     AS unique_cities,
        ROUND(AVG(order_amount), 2) AS avg_order_value,
        ROUND(SUM(order_amount), 2) AS total_revenue
    FROM {config['silver_orders_table']}
""").show()

# 2. Status distribution – sanity check
spark.sql(f"""
    SELECT order_status, COUNT(*) AS cnt,
           ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 1) AS pct
    FROM {config['silver_orders_table']}
    GROUP BY order_status
    ORDER BY cnt DESC
""").show()

# 3. City distribution
spark.sql(f"""
    SELECT city, COUNT(*) AS orders,
           ROUND(SUM(order_amount), 0) AS revenue
    FROM {config['silver_orders_table']}
    GROUP BY city
    ORDER BY revenue DESC
    LIMIT 10
""").show()

# 4. Data quality score distribution
spark.sql(f"""
    SELECT dq_score, COUNT(*) AS rows
    FROM {config['silver_orders_table']}
    GROUP BY dq_score
    ORDER BY dq_score DESC
""").show()

```


3.3 Silver Transformation for Customers and Products

Now do the same for the other two datasets. Create `nb_bronze_to_silver_customers` :

python

```
# =====  
# NOTEBOOK: nb_bronze_to_silver_customers  
# =====  
  
from pyspark.sql import functions as F  
from pyspark.sql import types as T  
from delta.tables import DeltaTable  
  
# Read customer CSV from Bronze  
customers_df = spark.read \  
    .option("header", "true") \  
    .option("inferSchema", "false") \  
    .csv("Files/raw/customers/")  
  
print(f"Raw customers: {customers_df.count():,}")  
customers_df.printSchema()  
  
# ——— Type casting ———  
  
typed_df = customers_df \  
    .withColumn("date_joined", F.to_date(F.col("date_joined"), "yyyy-MM-dd")) \  
    .withColumn("total_orders", F.col("total_orders").cast(T.IntegerType())) \  
    .withColumn("customer_id", F.trim(F.col("customer_id")))  
  
# ——— Standardize is_active (messy boolean) ———  
# Raw values: "Y", "y", "Yes", "1", "TRUE"  
# Canonical: boolean true/false  
def parse_bool_flag(val: str) -> bool:  
    if val is None:  
        return False  
    return val.strip().lower() in ("y", "yes", "1", "true", "t")  
  
parse_bool_udf = F.udf(parse_bool_flag, T.BooleanType())  
  
cleaned_df = typed_df \  
    .withColumn("is_active", parse_bool_udf(F.col("is_active"))) \  
    .withColumn(  
        # City standardization – same function as orders  
        "city",  
        # Using F.regex_replace for simple cases  
        F.when(F.lower(F.trim(F.col("city"))).isin(
```

```

        ["mumbai", "mum", "mbai", "bombay"]), F.lit("Mumbai")
    ).when(F.lower(F.trim(F.col("city"))).isin(
        ["delhi", "new delhi", "ncr"]), F.lit("Delhi")
    ).when(F.lower(F.trim(F.col("city"))).isin(
        ["bangalore", "bengaluru", "blr"]), F.lit("Bangalore")
    ).otherwise(F.initcap(F.trim(F.col("city"))))
) \
.withColumn(
    # Validate email: must contain @ and a dot after @
    "is_valid_email",
    F.col("email").rlike(r"^[^@]+@[^@]+\.[^@]+$")
) \
.withColumn(
    # Business rule: negative total_orders is impossible → null them
    "total_orders",
    F.when(F.col("total_orders") < 0, F.lit(None))
    .otherwise(F.col("total_orders"))
) \
.withColumn(
    # Customer tenure in days
    "days_as_customer",
    F.datediff(F.current_date(), F.col("date_joined"))
) \
.withColumn(
    # Loyalty tier standardization
    "loyalty_tier",
    F.when(F.col("loyalty_tier").isin(
        ["Bronze", "Silver", "Gold", "Platinum"]), F.col("loyalty_tier")
    ).otherwise(F.lit("Bronze")) # default new customers to Bronze
) \
.withColumn("silver_insert_timestamp", F.current_timestamp())

# ——— Quality gate


---


good_df = cleaned_df.filter(
    F.col("customer_id").isNotNull() &
    F.col("date_joined").isNotNull()
)

print(f"Silver customers: {good_df.count():,} rows")

# ——— Write to Silver


---


TARGET = "silver_lakehouse.silver_customers"

good_df.write \
    .format("delta") \
    .mode("overwrite") \

```

```

.option("overwriteSchema", "true") \
.saveAsTable(TARGET)

print(f"Written to {TARGET}")

# ——— Quick validation

spark.sql(f"""
    SELECT
        COUNT(*)                AS total_customers,
        SUM(CAST(is_active AS INT)) AS active_customers,
        SUM(CAST(is_valid_email AS INT)) AS valid_emails,
        AVG(days_as_customer)      AS avg_tenure_days,
        loyalty_tier,
        COUNT(*)                AS tier_count
    FROM {TARGET}
    GROUP BY loyalty_tier
    ORDER BY tier_count DESC
""").show()

```

3.4 Delta Lake Time Travel – The Production Superpower

One of the most impressive features of Delta Lake that you will use constantly in production:

python

```

# =====
# NOTEBOOK CELL – Delta Lake Time Travel
# =====
# Scenario: Your Silver pipeline ran with a bug on Monday.
# It's now Wednesday. You need to see what the data looked like
# BEFORE the bad run so you can fix it.
# Delta Lake makes this trivial.

TARGET = "silver_lakehouse.silver_orders"

# ——— View the change history

print("Delta table transaction history:")
spark.sql(f"DESCRIBE HISTORY {TARGET}").show(10, truncate=60)

# ——— Query data as of a specific version ———
# VERSION AS OF 0 = the very first write (initial load)
spark.sql(f"""
    SELECT COUNT(*), MAX(order_date)
    FROM {TARGET} VERSION AS OF 0
""").show()

```

```

# ——— Query data as of a specific timestamp ———
spark.sql(f"""
    SELECT COUNT(*), MAX(order_date)
    FROM {TARGET} TIMESTAMP AS OF '2024-01-15 08:00:00'
""").show()

# ——— ROLLBACK: restore table to a previous version ———
# Use this if today's pipeline run corrupted your Silver table
# Uncomment to actually roll back:
# spark.sql(f"""
#     RESTORE TABLE {TARGET} TO VERSION AS OF 2
# """)
# print("Table restored to version 2!")

# ——— VACUUM: clean up old versions to save storage ———
# Delta keeps ALL versions by default (unlimited storage growth)
# VACUUM removes versions older than the retention period
# Default retention: 7 days – never go below this!
spark.sql(f"VACUUM {TARGET} RETAIN 168 HOURS") # 168 = 7 days
print("Vacuum complete. Old versions older than 7 days removed.")

```

Time travel is one of the most asked-about Delta Lake features in interviews. Know it cold:

VERSION AS OF , **TIMESTAMP AS OF** , **RESTORE TABLE** , **DESCRIBE HISTORY** , and **VACUUM** . Know WHY VACUUM has a 7-day minimum (active Spark jobs could be reading old versions — VACUUM below 7 days could corrupt running jobs).

3.5 Optimizing Delta Tables — Z-Ordering

After a few weeks of data, your Silver table has many small Parquet files. Queries slow down because Spark opens hundreds of tiny files instead of a few large ones. Fix this with OPTIMIZE and Z-ORDER.

```

python

# =====
# CELL – OPTIMIZE and Z-ORDER (run weekly in production)
# =====

TARGET = "silver_lakehouse.silver_orders"

# OPTIMIZE: compacts many small files into fewer larger files
# This dramatically reduces "small file problem" overhead
print("Running OPTIMIZE – compacting small files...")
spark.sql(f"OPTIMIZE {TARGET}")

# Z-ORDER: co-locates related data within files

```

```

# Queries that filter by city AND order_date will be 10-50x faster
# because the relevant data is physically co-located in the same files
# Rule: Z-ORDER by the columns you filter/join on most frequently
print("Running Z-ORDER by order_date and city...")
spark.sql(f"""
    OPTIMIZE {TARGET}
    ZORDER BY (order_date, city)
""")
print("Optimization complete!")

# Check table stats after optimization
spark.sql(f"DESCRIBE DETAIL {TARGET}").select(
    "numFiles", "sizeInBytes", "partitionColumns"
).show(truncate=False)

```

Z-ORDER is another key interview topic. Explain it as: "co-locating similar values in the same file so the query engine can skip entire files when filtering." It's Spark's equivalent of a database index, but for columnar Parquet files.

3.6 Phase 3 — Common Mistakes and Interview Points

Phase 3 summary — concepts you must know

Schema enforcement

Always define schema explicitly — never inferSchema in prod.
PERMISSIVE mode vs FAILFAST vs DROP

Deduplication

Window + row_number
partitionBy key,
orderBy timestamp desc,
keep row_num == 1
= keep latest record

Data quality gate

Score each row
Route bad rows to quarantine table.
Abort pipeline if pass rate too low

Delta Lake features

MERGE = upsert
Time travel
OPTIMIZE + ZORDER
VACUUM (7d min)
DESCRIBE HISTORY

UDFs vs built-ins

Built-in Spark funcs run on JVM — fast.
Python UDFs serialize to Python — slower.
Use UDFs only when

Partitioning

Partition by date for time-series data.
Never partition by high-cardinality cols (order_id = disaster)

Top interview questions from Phase 3

1. "What is the difference between MERGE, INSERT OVERWRITE, and append in Delta Lake?"
2. "How do you handle late-arriving data in a batch pipeline?"
3. "What is the small file problem and how does OPTIMIZE solve it?"
4. "Walk me through your data quality framework — how do you handle bad rows?"

Phase 4 — Data Modeling and the Gold Layer (Star Schema)

Now your Silver data is clean and trusted. Phase 4 transforms it from clean transactional records into an **analytical model** that answers business questions in milliseconds.

4.1 What Is a Star Schema and Why Does Every Company Use It?

What is it?

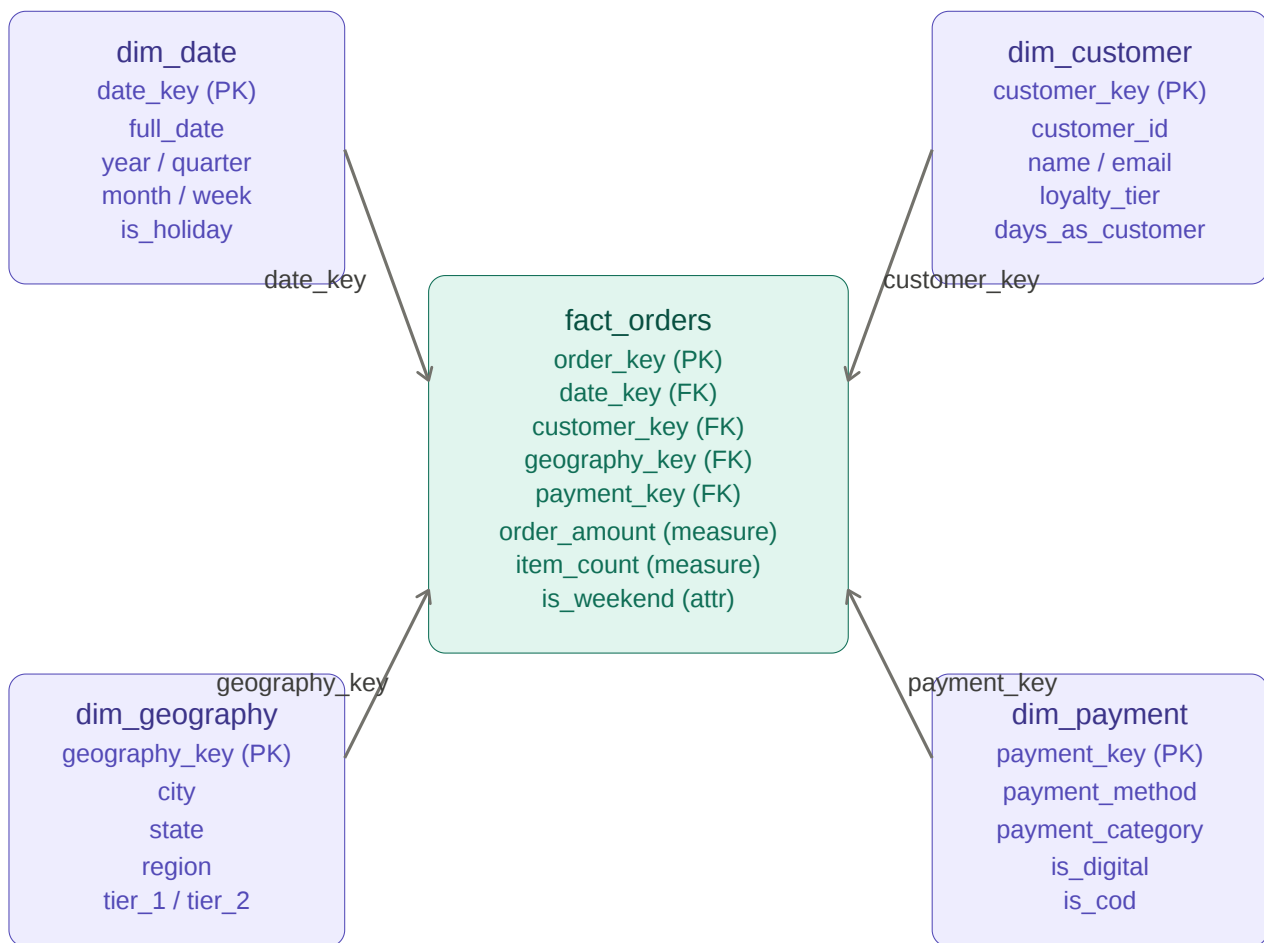
A star schema organizes data into two types of tables: **Fact tables** (the measurements — sales amounts, quantities, durations) and **Dimension tables** (the context — who, what, where, when). When you draw it, the fact table sits in the center surrounded by dimension tables — it looks like a star.

Why do companies use it over just querying Silver?

Querying Silver directly	Star schema Gold layer
Analyst writes 40-line SQL joining 6 tables	Analyst writes 8-line SQL joining 2 tables
Same join logic rewritten by 10 different analysts	Join logic defined once by data engineer
Report takes 45 seconds	Report takes 2 seconds (pre-aggregated)
Revenue calculation inconsistent across teams	Revenue defined once in fact_orders
Power BI relationship setup complex	Power BI relationships are automatic from schema

The RetailPulse Star Schema

RetailPulse Gold — star schema



4.2 Building the Gold Layer — Complete Notebook

Create `nb_silver_to_gold_star_schema` :

python

```
# =====  
# CELL 1 - Build dim_date (Date Dimension)  
# =====  
# dim_date is special: it's NOT derived from your data.  
# It's generated to cover a fixed date range.  
# Every analytics platform has one - it's how you get  
# "day of week", "quarter", "is holiday" without calculating  
# it in every single query.  
  
from pyspark.sql import functions as F  
from pyspark.sql import types as T  
from delta.tables import DeltaTable  
import pandas as pd  
from datetime import date, timedelta  
  
# Generate dates: 2020-01-01 to 2027-12-31 (covers history + future)
```

```

start_date = date(2020, 1, 1)
end_date   = date(2027, 12, 31)

date_list = []
current = start_date
while current <= end_date:
    date_list.append({
        "date_key":      int(current.strftime("%Y%m%d")), # 20240115
        "full_date":     current,
        "year":          current.year,
        "quarter":       (current.month - 1) // 3 + 1,
        "month_num":     current.month,
        "month_name":    current.strftime("%B"),          # "January"
        "month_short":   current.strftime("%b"),          # "Jan"
        "week_of_year":  current.isocalendar()[1],
        "day_of_month":  current.day,
        "day_of_week":   current.isoweekday(),            # 1=Mon, 7=Sun
        "day_name":      current.strftime("%A"),          # "Monday"
        "day_short":     current.strftime("%a"),          # "Mon"
        "is_weekend":    current.isoweekday() >= 6,
        "is_month_start": current.day == 1,
        "is_month_end":  (current + timedelta(days=1)).day == 1,
        "is_quarter_start": current.day == 1 and current.month in [1, 4, 7, 10],
        "fiscal_year":   current.year if current.month >= 4 else current.year - 1,
        "fiscal_quarter": ((current.month - 4) % 12) // 3 + 1,
        # Indian fiscal year starts April 1
        "year_month":    int(current.strftime("%Y%m")),   # 202401
    })
    current += timedelta(days=1)

# Indian public holidays 2024 (you'd load this from a reference file in production)
indian_holidays_2024 = {
    date(2024, 1, 26): "Republic Day",
    date(2024, 8, 15): "Independence Day",
    date(2024, 10, 2): "Gandhi Jayanti",
    date(2024, 11, 1): "Diwali",
}

dim_date_pd = pd.DataFrame(date_list)
dim_date_df = spark.createDataFrame(dim_date_pd)

# Add holiday info
holidays_df = spark.createDataFrame(
    [(int(k.strftime("%Y%m%d")), v) for k, v in indian_holidays_2024.items()],
    ["date_key", "holiday_name"]
)

dim_date_df = dim_date_df \
    .join(holidays_df, on="date_key", how="left") \

```



```

        .withColumn("is_holiday", F.col("holiday_name").isNotNull())

dim_date_df.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("gold_lakehouse.dim_date")

print(f"dim_date: {dim_date_df.count():,} rows")
spark.sql("SELECT * FROM gold_lakehouse.dim_date WHERE year=2024 LIMIT 5").show()

```

python

```

# =====
# CELL 2 – Build dim_customer (SCD Type 1)
# =====
# SCD = Slowly Changing Dimension
# Type 1: Overwrite the record. No history kept.
#         Use when history doesn't matter (e.g. loyalty tier)
# Type 2: Add a new row. Full history preserved.
#         Use when history matters (e.g. address changes for delivery analytics)
# We use Type 1 here – simpler and sufficient for our use case.

dim_customer_df = spark.sql("""
    SELECT
        -- Surrogate key: never expose source system IDs directly to BI
        -- Use monotonically_increasing_id as a stable surrogate
        CAST(ROW_NUMBER() OVER (ORDER BY customer_id) AS BIGINT) AS customer_key,

        customer_id,                -- natural/business key
        first_name,
        last_name,
        CONCAT(first_name, ' ', last_name) AS full_name,

        -- Mask PII – analysts don't need real emails
        -- In production, apply column-level security instead
        CASE WHEN is_valid_email THEN email ELSE 'invalid@email.com' END AS email,

        city,
        state,
        loyalty_tier,
        date_joined,
        days_as_customer,
        total_orders,
        is_active,

        -- Derived segments for analytics
    CASE

```

```

        WHEN days_as_customer > 730 THEN 'Long-term'
        WHEN days_as_customer > 180 THEN 'Established'
        WHEN days_as_customer > 30 THEN 'Growing'
        ELSE 'New'
    END AS customer_tenure_segment,

    CURRENT_TIMESTAMP() AS gold_insert_timestamp

FROM silver_lakehouse.silver_customers
WHERE customer_id IS NOT NULL
""")

dim_customer_df.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("gold_lakehouse.dim_customer")

print(f"dim_customer: {dim_customer_df.count():,} rows")

```

python

```

# =====
# CELL 3 - Build dim_geography
# =====

dim_geography_df = spark.sql("""
    SELECT
        ROW_NUMBER() OVER (ORDER BY city, state) AS geography_key,
        city,
        state,
        CASE state
            WHEN 'MH' THEN 'Maharashtra'
            WHEN 'DL' THEN 'Delhi'
            WHEN 'KA' THEN 'Karnataka'
            WHEN 'TS' THEN 'Telangana'
            WHEN 'TN' THEN 'Tamil Nadu'
            WHEN 'GJ' THEN 'Gujarat'
            WHEN 'RJ' THEN 'Rajasthan'
            WHEN 'WB' THEN 'West Bengal'
            ELSE 'Other'
        END AS state_full_name,
        CASE state
            WHEN 'MH' THEN 'West'
            WHEN 'DL' THEN 'North'
            WHEN 'KA' THEN 'South'
            WHEN 'TS' THEN 'South'
            WHEN 'TN' THEN 'South'

```

```

        WHEN 'GJ' THEN 'West'
        WHEN 'RJ' THEN 'North'
        WHEN 'WB' THEN 'East'
        ELSE 'Other'
    END AS region,
    CASE city
        WHEN 'Mumbai' THEN 'Tier 1'
        WHEN 'Delhi' THEN 'Tier 1'
        WHEN 'Bangalore' THEN 'Tier 1'
        WHEN 'Hyderabad' THEN 'Tier 1'
        WHEN 'Chennai' THEN 'Tier 1'
        WHEN 'Kolkata' THEN 'Tier 1'
        ELSE 'Tier 2'
    END AS city_tier
FROM (
    SELECT DISTINCT city, state
    FROM silver_lakehouse.silver_orders
    WHERE city != 'Unknown'
)
)"""

dim_geography_df.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("gold_lakehouse.dim_geography")

print(f"dim_geography: {dim_geography_df.count():,} rows")
dim_geography_df.show()

```

python

```

# =====
# CELL 4 - Build dim_payment
# =====

dim_payment_df = spark.sql("""
    SELECT
        ROW_NUMBER() OVER (ORDER BY payment_method) AS payment_key,
        payment_method,
        CASE payment_method
            WHEN 'UPI' THEN 'Digital'
            WHEN 'Credit Card' THEN 'Card'
            WHEN 'Debit Card' THEN 'Card'
            WHEN 'Net Banking' THEN 'Digital'
            WHEN 'Wallet' THEN 'Digital'
            WHEN 'Cash on Delivery' THEN 'Cash'
            ELSE 'Other'

```

```

        END AS payment_category,
        payment_method IN ('UPI', 'Credit Card', 'Debit Card',
                           'Net Banking', 'Wallet') AS is_digital,
        payment_method = 'Cash on Delivery' AS is_cod,
        payment_method IN ('Credit Card', 'Debit Card') AS is_card
    FROM (
        SELECT DISTINCT payment_method
        FROM silver_lakehouse.silver_orders
        WHERE payment_method IS NOT NULL
    )
    """

dim_payment_df.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("gold_lakehouse.dim_payment")

print(f"dim_payment: {dim_payment_df.count():,} rows")
dim_payment_df.show()

```

python

```

# =====
# CELL 5 – Build fact_orders (The Central Fact Table)
# =====
# This is the most important table in your entire data platform.
# It joins Silver orders with all dimension surrogate keys.
# ALL measures (revenue, count, items) live here.
# ALL filters (by city, by date, by customer) go through dimensions.

fact_orders_df = spark.sql("""
    SELECT
        -- Surrogate key for the fact row itself
        MONOTONICALLY_INCREASING_ID() AS order_key,

        -- Foreign keys to dimensions (integers, not strings = fast joins)
        dd.date_key,
        dc.customer_key,
        dg.geography_key,
        dp.payment_key,

        -- Business / natural key (keep for lineage and debugging)
        so.order_id,
        so.customer_id,

        -- MEASURES: the numeric facts being analyzed
        so.order_amount AS gross_revenue,

```

```

so.item_count,

-- Derived measures (business-defined)
CASE WHEN so.order_status = 'returned'
      THEN so.order_amount ELSE 0 END AS returned_amount,
CASE WHEN so.order_status = 'cancelled'
      THEN 1 ELSE 0 END AS is_cancelled,
CASE WHEN so.order_status = 'delivered'
      THEN so.order_amount ELSE 0 END AS net_revenue,
CASE WHEN so.has_discount
      THEN 1 ELSE 0 END AS has_discount_flag,

-- Attributes on the fact (low-cardinality flags only)
so.order_status,
so.order_size_tier,
so.is_weekend,
so.is_guest_order,
so.dq_score,

-- Lineage
CURRENT_TIMESTAMP() AS gold_insert_timestamp

FROM silver_lakehouse.silver_orders so

-- Join to date dimension on date key integer (YYYYMMDD)
JOIN gold_lakehouse.dim_date dd
  ON dd.date_key = CAST(DATE_FORMAT(so.order_date, 'yyyyMMdd') AS INT)

-- Join to customer dimension on customer_id
LEFT JOIN gold_lakehouse.dim_customer dc
  ON dc.customer_id = so.customer_id

-- Join to geography on city + state
LEFT JOIN gold_lakehouse.dim_geography dg
  ON dg.city = so.city AND dg.state = so.state

-- Join to payment dimension
LEFT JOIN gold_lakehouse.dim_payment dp
  ON dp.payment_method = so.payment_method

-- Only include completed/active orders in Gold fact
-- Cancelled orders ARE included – they're needed for cancellation rate KPI
WHERE so.order_id IS NOT NULL
      AND so.order_amount IS NOT NULL
""")

fact_orders_df.write \
  .format("delta") \

```

```
.mode("overwrite") \
.partitionBy("date_key") \
.saveAsTable("gold_lakehouse.fact_orders")
```

```
row_count = spark.sql("SELECT COUNT(*) FROM gold_lakehouse.fact_orders").collect()[0][0]
print(f"fact_orders: {row_count:,} rows")
```

python

```
# =====
# CELL 6 – Build agg_daily_sales (Pre-aggregated Gold table)
# =====
# For the executive dashboard – answers "what was today's revenue?"
# in under 1 second because ALL aggregation is pre-computed.
```

```
spark.sql("""
CREATE OR REPLACE TABLE gold_lakehouse.agg_daily_sales
USING DELTA AS

SELECT
    dd.full_date,
    dd.year,
    dd.quarter,
    dd.month_num,
    dd.month_name,
    dd.day_name,
    dd.is_weekend,
    dd.is_holiday,
    dd.holiday_name,
    dg.city,
    dg.state_full_name,
    dg.region,
    dg.city_tier,

    -- Revenue metrics
    SUM(fo.gross_revenue)      AS total_gross_revenue,
    SUM(fo.net_revenue)       AS total_net_revenue,
    SUM(fo.returned_amount)   AS total_returns,

    -- Volume metrics
    COUNT(fo.order_key)       AS total_orders,
    SUM(fo.item_count)        AS total_items,
    SUM(fo.is_cancelled)      AS cancelled_orders,

    -- Averages
    AVG(fo.gross_revenue)     AS avg_order_value,
```

```

-- Rate metrics
ROUND(SUM(fo.is_cancelled) * 100.0 / COUNT(*), 2) AS cancellation_rate_pct,
ROUND(SUM(CASE WHEN fo.returned_amount > 0 THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS
return_rate_pct,
ROUND(SUM(fo.has_discount_flag) * 100.0 / COUNT(*), 2) AS discount_rate_pct

FROM gold_lakehouse.fact_orders fo
JOIN gold_lakehouse.dim_date dd ON dd.date_key = fo.date_key
JOIN gold_lakehouse.dim_geography dg ON dg.geography_key = fo.geography_key

GROUP BY
    dd.full_date, dd.year, dd.quarter, dd.month_num, dd.month_name,
    dd.day_name, dd.is_weekend, dd.is_holiday, dd.holiday_name,
    dg.city, dg.state_full_name, dg.region, dg.city_tier
""")

print("agg_daily_sales created!")
row_count = spark.sql("SELECT COUNT(*) FROM gold_lakehouse.agg_daily_sales").collect()[0][0]
print(f"agg_daily_sales: {row_count:,} rows")

# Quick executive summary
spark.sql("""
    SELECT
        month_name,
        SUM(total_gross_revenue) AS monthly_revenue,
        SUM(total_orders) AS monthly_orders,
        AVG(avg_order_value) AS avg_order_value,
        AVG(cancellation_rate_pct) AS avg_cancel_rate
    FROM gold_lakehouse.agg_daily_sales
    GROUP BY month_name, month_num
    ORDER BY month_num
""").show()

```

4.3 Querying Gold with the SQL Analytics Endpoint

Now go to your `gold_lakehouse` in the Fabric portal. At the top-right, click **SQL analytics endpoint**. You're now in a SQL query environment — no Spark required.

Try these business queries:

```

sql

-- Query 1: Top cities by revenue this month
SELECT
    dg.city,
    dg.region,
    dg.city_tier,

```

```

COUNT(fo.order_key)          AS total_orders,
SUM(fo.gross_revenue)         AS total_revenue,
AVG(fo.gross_revenue)         AS avg_order_value,
SUM(fo.is_cancelled)          AS cancellations
FROM gold_lakehouse.fact_orders fo
JOIN gold_lakehouse.dim_date   dd ON dd.date_key      = fo.date_key
JOIN gold_lakehouse.dim_geography dg ON dg.geography_key = fo.geography_key
WHERE dd.year = 2024 AND dd.month_num = 1
GROUP BY dg.city, dg.region, dg.city_tier
ORDER BY total_revenue DESC;

-- Query 2: Weekend vs weekday performance
SELECT
    is_weekend,
    COUNT(*)          AS orders,
    SUM(gross_revenue) AS revenue,
    AVG(gross_revenue) AS avg_order_value
FROM gold_lakehouse.fact_orders
GROUP BY is_weekend;

-- Query 3: Payment method adoption
SELECT
    dp.payment_method,
    dp.payment_category,
    dp.is_digital,
    COUNT(fo.order_key)          AS order_count,
    ROUND(COUNT(fo.order_key) * 100.0 / SUM(COUNT(*)) OVER (), 1) AS pct_share,
    SUM(fo.gross_revenue)        AS revenue
FROM gold_lakehouse.fact_orders fo
JOIN gold_lakehouse.dim_payment dp ON dp.payment_key = fo.payment_key
GROUP BY dp.payment_method, dp.payment_category, dp.is_digital
ORDER BY order_count DESC;

```

4.4 Phase 4 — What's Next

Your Gold layer is now complete:

- `dim_date` , `dim_customer` , `dim_geography` , `dim_payment` — all dimension tables
- `fact_orders` — the central fact table with all measures and foreign keys
- `agg_daily_sales` — pre-aggregated for executive dashboards

Phase 5 (Semantic Model + Power BI) will take this Gold layer and build the business-facing reporting layer on top of it — adding DAX measures, row-level security, and interactive dashboards.

Phase 6 (Orchestration) will chain all four notebooks (ingest → bronze-to-silver → silver-to-gold) into a single scheduled pipeline that runs every morning at 6am automatically.

Type **"continue to Phase 5"** to build the Power BI Semantic Model and executive dashboard, or ask any questions about what we built in Phases 3 and 4 before moving forward.

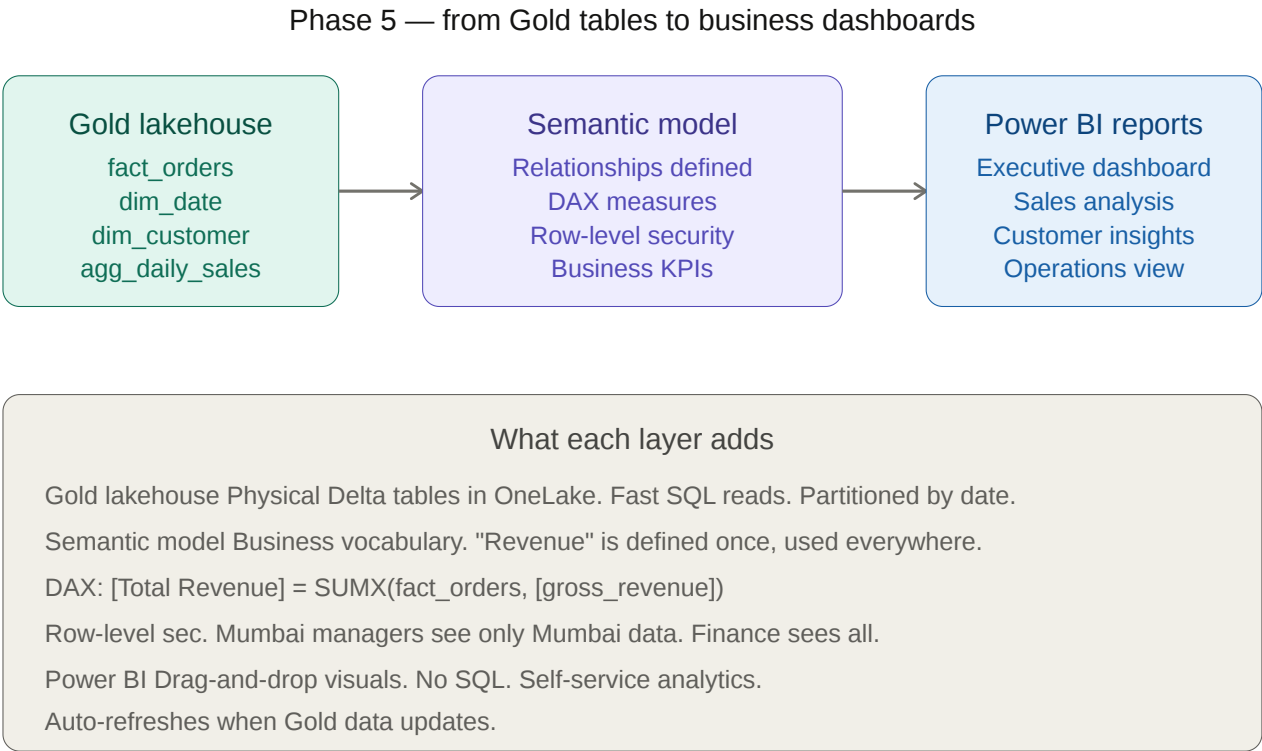
2026-06-14 10:51:29 You Asked 

continue in proper guide



Phase 5 – Semantic Model & Power BI Reporting

Now let's build the complete reporting layer – the part the business actually sees and uses every day.



5.1 What Is a Semantic Model and Why Does Every Enterprise Need One?

What is it?

A Semantic Model (previously called a Power BI Dataset) is a **business translation layer** that sits between your Gold tables and your report consumers. It defines:

- Which tables exist and how they relate to each other
- What every business metric means and how to calculate it

- Who is allowed to see which data
- What the "friendly names" for technical columns are

Why do companies use it instead of connecting Power BI directly to tables?

Without a semantic model, every analyst who builds a report must manually figure out how to join `fact_orders` to `dim_date`, what "net revenue" means (gross minus returns?), and which cities they're allowed to see. Ten analysts will write ten different formulas and get ten different revenue numbers.

With a semantic model, all of that is defined once by the data engineering team. Every report that uses this semantic model automatically uses the same definitions, the same security rules, and the same relationships. It is the **single source of truth for business logic**.

Real-world analogy

A semantic model is like a company's official accounting glossary. Every department has their own spreadsheets, but the CFO has defined exactly what "revenue," "EBITDA," and "working capital" mean. Everyone who uses those words is referring to the same official definition. No ambiguity, no disputes.

5.2 Creating the Semantic Model — Step-by-Step Guide

Step 1 — Open the Semantic Model from Your Gold Lakehouse

1. Go to your `gold_lakehouse` in your workspace
2. At the top right, click **SQL analytics endpoint**
3. In the SQL endpoint view, click **Reporting** tab at the top
4. Click **New semantic model**
5. Name it: `sm_retailpulse_analytics`
6. Select these tables to include:
 - `fact_orders`
 - `dim_date`
 - `dim_customer`
 - `dim_geography`
 - `dim_payment`
 - `agg_daily_sales`
7. Click **Confirm**

Fabric automatically creates the semantic model and opens the **Model view** where you see your tables laid out as boxes.

Step 2 – Define Relationships Between Tables

Fabric may auto-detect some relationships, but you must verify every single one. In the Model view:

Creating the fact_orders → dim_date relationship:

1. Drag `date_key` from `fact_orders` onto `date_key` in `dim_date`
2. A relationship dialog opens. Set:
 - Cardinality: **Many to one** (`fact_orders` has many rows per date, `dim_date` has one row per date)
 - Cross filter direction: **Single** (dim filters fact, not the other way)
 - Make this relationship active: **Yes** (checked)
3. Click **Save**

Cardinality is a critical concept. Many-to-one (`*:1`) means the "many" side (fact table) can have duplicate values in the join column, but the "one" side (dimension) must have unique values. If your `dim_date` has duplicate `date_key` values, Power BI will error. This is why surrogate keys in dimensions must always be unique.

Repeat for all relationships:

From table	From column	To table	To column	Cardinality
<code>fact_orders</code>	<code>date_key</code>	<code>dim_date</code>	<code>date_key</code>	Many to one
<code>fact_orders</code>	<code>customer_key</code>	<code>dim_customer</code>	<code>customer_key</code>	Many to one
<code>fact_orders</code>	<code>geography_key</code>	<code>dim_geography</code>	<code>geography_key</code>	Many to one
<code>fact_orders</code>	<code>payment_key</code>	<code>dim_payment</code>	<code>payment_key</code>	Many to one

Why no relationship for `agg_daily_sales` ? Because it is a pre-aggregated table — it already has `city` and `full_date` as regular columns, not foreign keys. Reports that use `agg_daily_sales` don't need to join anything. It's designed for the fastest possible dashboard loads.

Step 3 – Mark the Date Table (Critical Step Most Beginners Miss)

1. Click on the `dim_date` table in the model
2. In the top ribbon, click **Table tools** → **Mark as date table**
3. Select `full_date` as the date column
4. Click **Validate** → it should confirm no duplicate or null dates

Why must you mark the date table? Without this, Power BI's built-in time intelligence functions (`TOTALYTD` , `SAMEPERIODLASTYEAR` , `DATEADD`) don't work correctly. They need to know

which table is the authoritative date dimension. If your year-over-year comparison gives wrong results, this is almost always the reason.

Step 4 – Hide Technical Columns From Report Users

Your fact table has columns like `date_key` (an integer like `20240115`) that analysts should never use in reports — they should use `dim_date.full_date` instead. Hide the technical columns:

1. In the `fact_orders` table, right-click `date_key` → **Hide in report view**
2. Repeat for: `customer_key`, `geography_key`, `payment_key`, `order_key`
3. In `dim_customer`, hide `customer_key`
4. In `dim_date`, hide `date_key`, `year_month`

This is a UX discipline. When an analyst opens the field list in Power BI, they should see clean business fields like `Order Date`, `Customer Name`, `City` — not `date_key`, `geography_key`, `silver_insert_timestamp`. Senior data engineers think carefully about what report users see.

Step 5 – Rename Columns to Business-Friendly Names

In each table, click column headers to rename them:

Table	Technical name	Business name
<code>fact_orders</code>	<code>gross_revenue</code>	Gross Revenue
<code>fact_orders</code>	<code>net_revenue</code>	Net Revenue
<code>fact_orders</code>	<code>is_cancelled</code>	Is Cancelled
<code>fact_orders</code>	<code>order_id</code>	Order ID
<code>dim_customer</code>	<code>full_name</code>	Customer Name
<code>dim_customer</code>	<code>loyalty_tier</code>	Loyalty Tier
<code>dim_date</code>	<code>full_date</code>	Order Date
<code>dim_date</code>	<code>month_name</code>	Month
<code>dim_date</code>	<code>day_name</code>	Day of Week
<code>dim_geography</code>	<code>city_tier</code>	City Tier
<code>dim_geography</code>	<code>state_full_name</code>	State

5.3 Writing DAX Measures – The Business Logic Layer

DAX (Data Analysis Expressions) is the formula language of Power BI and semantic models. This is where you encode business definitions. Every KPI that appears on an executive dashboard should be defined as a DAX measure — never as an ad-hoc field calculation in a report.

What is a DAX Measure vs a Calculated Column?

DAX Measure	Calculated Column
Calculated at query time	Calculated at data refresh time
Changes based on filters/slicers	Same value for every row
[Total Revenue] changes when you filter by city	order_size_tier is fixed per row
Stored in the semantic model	Stored in the table
Best for KPIs and aggregations	Best for row-level attributes

Creating DAX Measures – Step-by-Step

In the semantic model, click **New measure** in the ribbon:

dax

/*

MEASURE GROUP 1: REVENUE METRICS

These are the financial KPIs. The CFO cares about these.

Business rule: Revenue = sum of gross_revenue from all non-null orders

 $\ast/$

-- Measure 1: Total Gross Revenue

```
-- SUMX iterates every row in fact_orders and sums gross_revenue
```

-- We use SUMX instead of SUM because it handles complex row-level logic

Total Revenue =

```
SUM(
  fact_orders,
  fact_orders[Gross Revenue]
)
```

-- Measure 2: Net Revenue (after returns)

```
-- SUM is simpler here because net_revenue is already a column
```

Net Revenue =

```
SUM( fact_orders[net_revenue] )
```

-- Measure 3: Return Amount

```

Return Amount =
    SUM( fact_orders[returned_amount] )

-- Measure 4: Return Rate %
-- DIVIDE is safer than "/" - returns 0 instead of ERROR when denominator is 0
Return Rate % =
    DIVIDE(
        [Return Amount],
        [Total Revenue],
        0          -- alternate result if denominator is 0
    ) * 100

-- Measure 5: Average Order Value (AOV) - KEY retail KPI
Average Order Value =
    DIVIDE(
        [Total Revenue],
        [Total Orders],
        0
    )

```

dax

```

/*
=====
//  MEASURE GROUP 2: VOLUME METRICS  //
=====
*/

-- Total Orders: COUNTROWS counts every row in the fact table
-- After applying any active filters (city, date, etc.)
Total Orders =
    COUNTROWS( fact_orders )

-- Cancelled Orders
Cancelled Orders =
    CALCULATE(
        COUNTROWS( fact_orders ),
        fact_orders[order_status] = "cancelled"
    )

-- CALCULATE is the most important DAX function.
-- It evaluates an expression in a MODIFIED filter context.
-- Here: count rows WHERE order_status = "cancelled"

-- Cancellation Rate %
Cancellation Rate % =

```

```

DIVIDE(
    [Cancelled Orders],
    [Total Orders],
    0
) * 100

-- Unique Customers
Unique Customers =
    DISTINCTCOUNT( fact_orders[customer_id] )

-- Guest Order Rate %
Guest Order Rate % =
    DIVIDE(
        CALCULATE( COUNTROWS(fact_orders), fact_orders[is_guest_order] = TRUE() ),
        [Total Orders],
        0
    ) * 100

```

dax

```

/*
=====
//  MEASURE GROUP 3: TIME INTELLIGENCE – The Power of DAX  //
=====

These are what separate a basic report from an executive dashboard.
They only work because you marked dim_date as the date table.
*/

-- Revenue Last Month
Revenue Last Month =
    CALCULATE(
        [Total Revenue],
        DATEADD( dim_date[Order Date], -1, MONTH )
    )
-- DATEADD shifts the date context. When user selects Jan 2024,
-- this measure automatically calculates Dec 2023.

-- Month-over-Month Revenue Change %
MoM Revenue Change % =
    DIVIDE(
        [Total Revenue] - [Revenue Last Month],
        [Revenue Last Month],
        0
    ) * 100

```

```

-- Revenue Same Period Last Year (Year-over-Year)
Revenue SPLY =
    CALCULATE(
        [Total Revenue],
        SAMEPERIODLASTYEAR( dim_date[Order Date] )
    )

-- Year-over-Year Growth %
YoY Growth % =
    DIVIDE(
        [Total Revenue] - [Revenue SPLY],
        [Revenue SPLY],
        0
    ) * 100

-- Year-to-Date Revenue
Revenue YTD =
    TOTALYTD(
        [Total Revenue],
        dim_date[Order Date]
        -- Optional: add "3-31" for fiscal year ending March 31 (Indian FY)
        -- TOTALYTD([Total Revenue], dim_date[Order Date], "3-31")
    )

-- Running Total Revenue (cumulative, great for trend charts)
Revenue Running Total =
    CALCULATE(
        [Total Revenue],
        DATESYTD( dim_date[Order Date] )
    )

```

dax

```

/*
=====
//  MEASURE GROUP 4: CUSTOMER ANALYTICS  //
=====
*/

-- Revenue per Customer (LTV proxy)
Revenue per Customer =
    DIVIDE( [Total Revenue], [Unique Customers], 0 )

-- New vs Returning Customer Revenue Split
-- "New" = customer's first order ever

```



```

New Customer Revenue =
    CALCULATE(
        [Total Revenue],
        FILTER(
            dim_customer,
            dim_customer[customer_tenure_segment] = "New"
        )
    )

```

-- Digital Payment Adoption Rate

```

Digital Payment Rate % =
    DIVIDE(
        CALCULATE(
            [Total Orders],
            dim_payment[is_digital] = TRUE()
        ),
        [Total Orders],
        0
    ) * 100

```

-- UPI Dominance (key Indian market insight)

```

UPI Share % =
    DIVIDE(
        CALCULATE(
            [Total Orders],
            dim_payment[payment_method] = "UPI"
        ),
        [Total Orders],
        0
    ) * 100

```

After creating each measure, immediately test it. Click the measure → click **Quick measure** or create a simple table visual with just that measure. A measure that returns **BLANK** when it should return a number is almost always a relationship problem or a filter context issue.

5.4 Setting Up Row-Level Security (RLS)

What is RLS and why is it essential in production?

Row-Level Security restricts what data a user sees based on their identity. In RetailPulse:

- The Mumbai Regional Manager should only see Mumbai data
- The Finance team sees all data across all cities
- The CEO dashboard shows everything
- The customer service team sees only their own region's customers

Without RLS, you'd need to build 4 separate reports and maintain them all. With RLS, you build one report and define security rules — Power BI automatically filters the data based on who logs in.

Step-by-Step: Create RLS Roles

In the semantic model, go to **Modeling** tab → **Manage roles** → **Create**:

dax

```
-- ROLE 1: Mumbai Regional Manager
-- Name: Mumbai_Manager
-- Table: dim_geography
-- Filter expression:
[city] = "Mumbai"

-- ROLE 2: South Region Manager (covers Bangalore, Chennai, Hyderabad)
-- Name: South_Region_Manager
-- Table: dim_geography
-- Filter expression:
[region] = "South"

-- ROLE 3: Tier 1 Cities Only (investor-facing view)
-- Name: Tier1_Cities
-- Table: dim_geography
-- Filter expression:
[city_tier] = "Tier 1"

-- ROLE 4: Finance (Full Access)
-- Name: Finance_Full_Access
-- No filter – this role sees everything
-- Leave filter expression empty
```

Test Your RLS Before Publishing

Still in the semantic model:

1. Click **Modeling** → **View as** → select **Mumbai_Manager**
2. Create a quick table visual with **city** and **Total Revenue**
3. You should see ONLY Mumbai data, even though all cities exist in the table

RLS testing before publishing is non-negotiable. A security breach where a Mumbai manager sees Delhi data is a compliance and business trust disaster. Always test every role with

View as before any report goes to users.

Assigning Users to Roles

After publishing the semantic model:

1. Go to your workspace
2. Find `sm_retailpulse_analytics` → click three dots `...` → **Security**
3. Click `Mumbai_Manager` role
4. Add email addresses: `mumbai.manager@retailpulse.com`
5. Click **Add** → **Save**

5.5 Building the Executive Dashboard in Power BI

Step 1 — Create a New Report

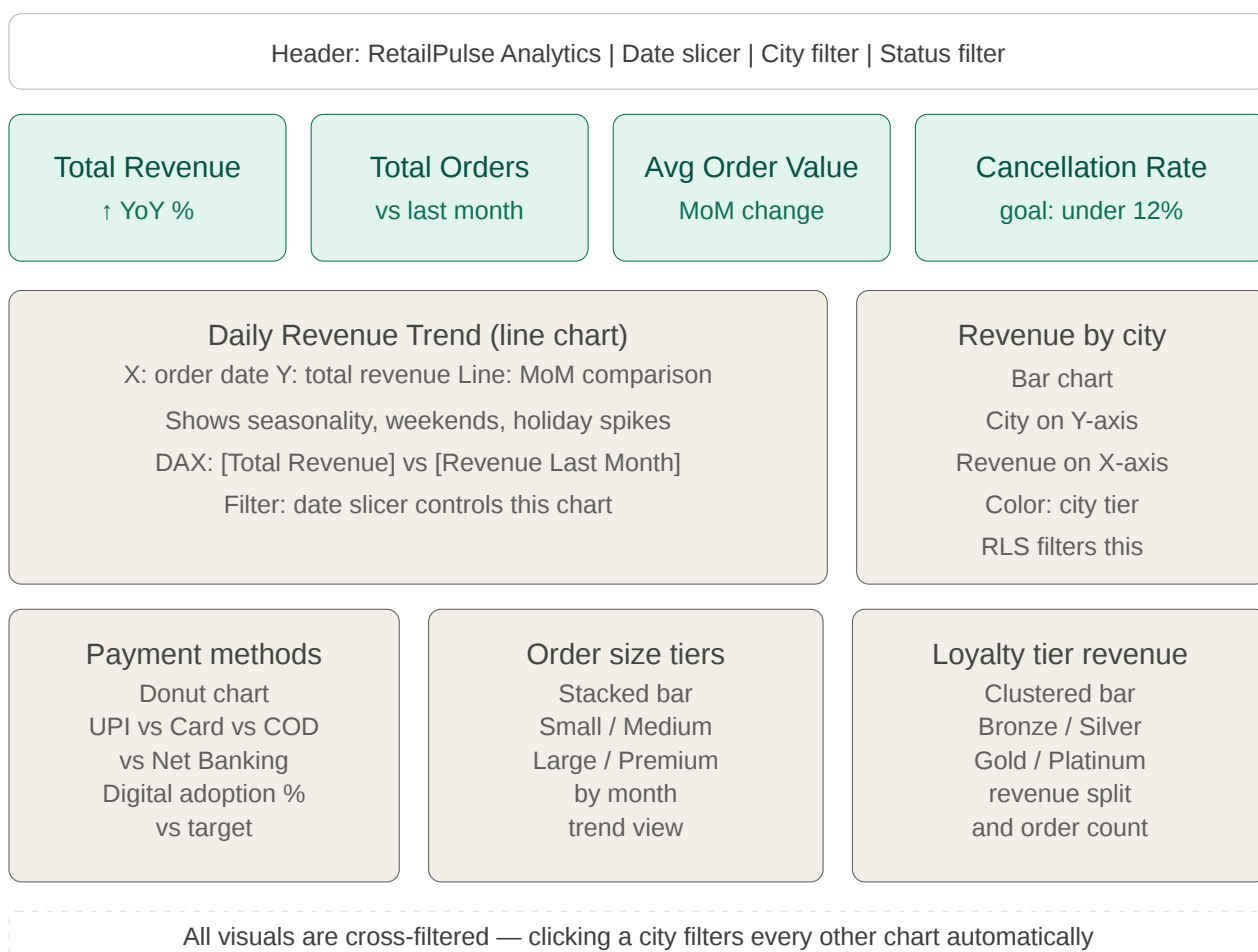
1. In your workspace, click **+ New item** → **Report**
2. Under "Pick a published semantic model", select `sm_retailpulse_analytics`
3. Click **Create**

You are now in the Power BI report canvas with your semantic model fields on the right panel.

Step 2 — Design the Report Layout

Before adding any visuals, set up the canvas layout. Think of it in zones:

RetailPulse executive dashboard — layout plan



[continue](#)

Phase 5 continued — Building the Dashboard, Then Phase 6 Orchestration

Let's finish the executive dashboard build with concrete click-by-click steps, then move into automating the entire pipeline end-to-end.

5.6 Building Each Visual — Step-by-Step

KPI Card 1 — Total Revenue

1. From the **Visualizations** pane, click the **Card** visual (the one showing a single big number)
2. Drag **Total Revenue** (your DAX measure) into the **Fields** well
3. In the **Format visual** pane (paint roller icon):
 - Callout value → Font size: **28** , Font: **bold**
 - Display units: **Auto** (so ₹4,52,30,000 shows as "₹4.52 Cr" if you set Indian format, or use Thousands/Millions)
 - Category label → toggle on, set text to "Total Revenue"
4. Resize the card to fit in the KPI row (use the layout from the wireframe above)

Why format display units? Showing "₹45230891.40" on a dashboard is unreadable and unprofessional. Executives scan dashboards in 3 seconds — large rounded numbers with clear labels communicate instantly. This is a basic but frequently overlooked production polish point.

Add a trend indicator using Conditional Formatting:

1. Click the card → **Format visual** → **Callout value**
2. Scroll to find an option to add a secondary comparison, or better, use a separate small **KPI visual** (not Card) which natively supports trend/target:
 - Visualizations → **KPI visual**
 - Indicator: **Total Revenue**
 - Trend axis: **dim_date[Order Date]**
 - Target: type a fixed value or another measure like **Revenue Last Month**

KPI Card 2, 3, 4 – Repeat the Pattern

Repeat the same Card/KPI visual creation for:

- Total Orders
- Average Order Value
- Cancellation Rate %

For Cancellation Rate, add conditional color formatting:

1. Click the card → **Format visual** → **Callout value** → **Conditional formatting**
2. Set rule: if value > 15 → red; if <= 12 → green; else → amber

This is called a "stoplight" pattern and it's extremely common in executive dashboards — it lets a CEO glance at a number and instantly know if it's good or bad without reading any text.

Chart 1 – Daily Revenue Trend (Line Chart)

1. Visualizations pane → click **Line chart**
2. X-axis: drag `dim_date[Order Date]`
3. Y-axis: drag `Total Revenue` (your measure)
4. To add the MoM comparison line: drag `Revenue Last Month` into Y-axis as a second value

Format the line chart:

1. Format visual → **X-axis** → Type: `Continuous` (treats dates as continuous time, not categories)
2. **Y-axis** → Display units: `Auto`
3. **Lines** → Make the "Revenue Last Month" line dashed and gray (so it reads as comparison, not primary)

Add a forecast (impressive production feature):

1. Click on the line chart → in the Visualizations **Analytics** pane (magnifying glass icon)
2. Click **Forecast** → **Add**
3. Set forecast length: `30 points` (30 days), Confidence interval: `95%`

This uses Power BI's built-in exponential smoothing algorithm to project future revenue based on historical patterns. It's a genuinely useful feature for planning and a great talking point in interviews — "I added trend forecasting to the executive dashboard using Power BI's native analytics."

Chart 2 – Revenue by City (Bar Chart)

1. Visualizations → **Clustered bar chart** (horizontal bars work better for city names than vertical columns)
2. Y-axis (categories): `dim_geography[city]`
3. X-axis (values): `Total Revenue`
4. Legend: `dim_geography[city_tier]` (colors Tier 1 vs Tier 2 differently)
5. Sort: click the three-dot menu on the visual → **Sort by** → `Total Revenue` → Descending

Chart 3 – Payment Method Distribution (Donut Chart)

1. Visualizations → **Donut chart**
2. Legend: `dim_payment[payment_method]`
3. Values: `Total Orders`
4. Format → **Data labels** → on, label content: `Category, Percentage of total`

Chart 4 – Order Size Tier Trend (Stacked Area or Column)

1. Visualizations → **Stacked column chart**
2. X-axis: `dim_date[month_name]`
3. Y-axis: `Total Orders`
4. Legend: `fact_orders[order_size_tier]`

Chart 5 – Loyalty Tier Revenue (Clustered Column)

1. Visualizations → **Clustered column chart**
2. X-axis: `dim_customer[loyalty_tier]`
3. Y-axis: `Total Revenue` and `Total Orders` (two measures, dual display)

Step 3 – Add Slicers (Interactive Filters)

1. Visualizations → **Slicer**
2. Field: `dim_date[Order Date]` → Format as a date range slider
3. Add a second slicer: `dim_geography[city]` → Format as a dropdown list
4. Add a third slicer: `fact_orders[order_status]`

Place these at the top of the report so users can filter the entire page at once.

Step 4 – Add a Page for Customer Analytics

1. Right-click the page tab at the bottom → **New page**
2. Name it: `Customer Insights`

3. Build visuals using `dim_customer` fields: loyalty tier distribution, customer tenure segments, new vs returning revenue split (using your `New Customer Revenue` measure)

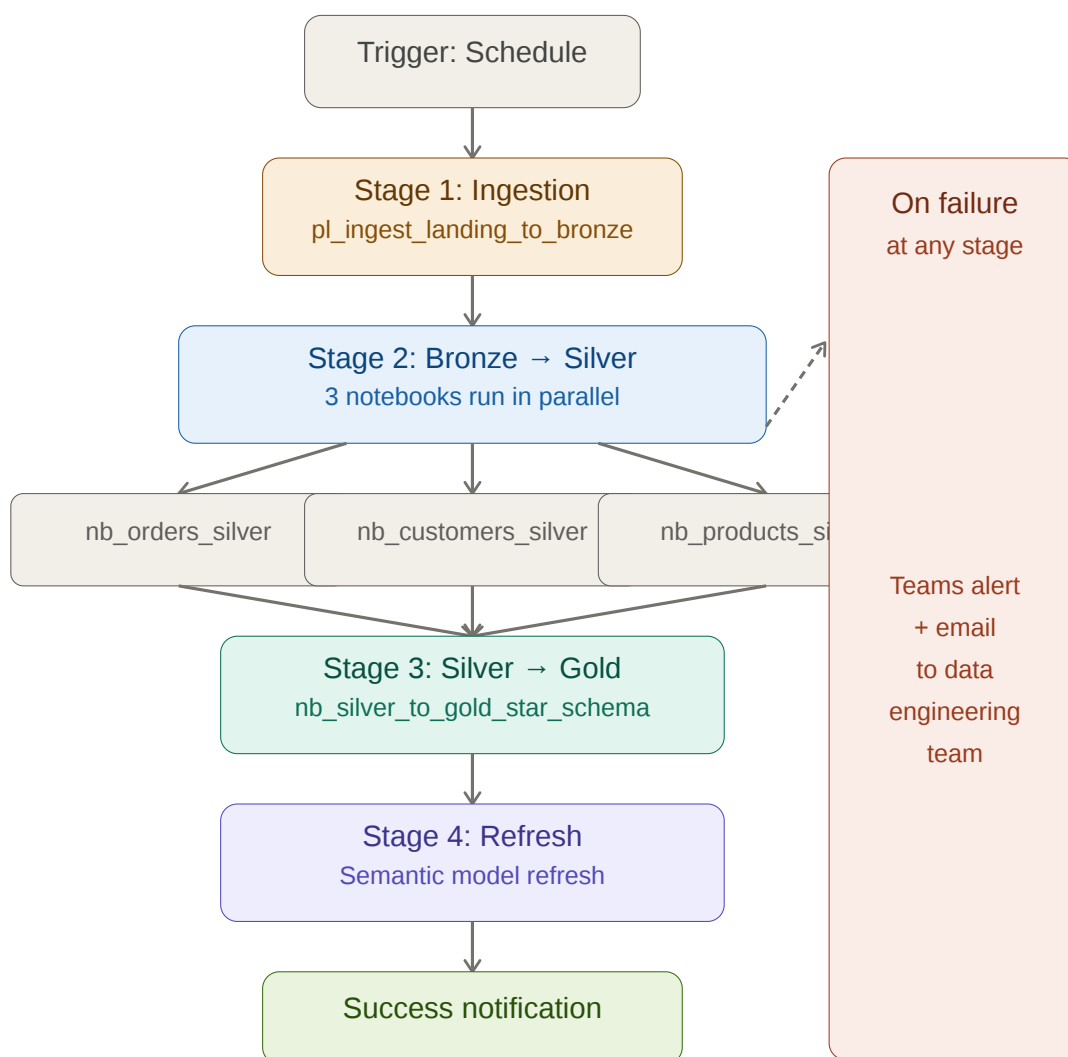
Step 5 — Publish and Test

1. Click **File** → **Save** → name the report `RetailPulse Executive Dashboard`
2. It's saved directly to your workspace (no separate "publish" step needed in Fabric — it's already there)
3. Test RLS: click **File** → ****... **** or use the workspace item options → **View as roles** if available, or simply verify by switching your own access

Phase 6 — Orchestration and Automation

Now we connect every piece you've built into one automated system that runs every morning without anyone touching a button.

`pl_master_daily_refresh` — runs every day at 6:00 AM IST



6.1 What Is Orchestration and Why Is It the Final Piece?

What is it?

Orchestration is the discipline of chaining individual data engineering steps (ingest → clean → model → refresh reports) into a single automated, monitored, schedule-driven workflow. Every piece you've built so far — the ingestion pipeline, three Silver notebooks, the Gold notebook, the semantic model — currently must be run manually, one at a time, by you clicking "Run". That is not production. Production means it runs itself, every single day, without anyone touching it.

Why do companies invest heavily in orchestration?

Imagine RetailPulse's data team manually ran 6 different jobs every morning at 5am. What happens when an engineer is on leave? Sick? It's a public holiday? The dashboard goes stale and the CEO notices immediately. Orchestration removes the human dependency entirely. It also gives you something manual processes never give you: a complete, auditable history of every single run, what succeeded, what failed, and why.

Why use a master Pipeline instead of scheduling each notebook separately?

Scheduling notebooks individually	Master orchestration pipeline
No guarantee Silver runs only after Bronze finishes	Explicit dependency: Silver waits for Bronze to succeed
If Bronze fails, Silver runs anyway on stale data	Pipeline halts the chain if any stage fails
No single place to see "did today's full run succeed?"	One pipeline run = one clear pass/fail for the entire day
Notebooks scheduled at fixed times can race each other	Pipeline enforces sequential or parallel execution rules explicitly

6.2 Building the Master Orchestration Pipeline — Step by Step

Step 1 — Create the Master Pipeline

1. In your workspace, click **+ New item** → **Data pipeline**
2. Name: `pl_master_daily_refresh`
3. Click **Create**

Step 2 — Add the Ingestion Pipeline as the First Activity

1. From the Activities toolbar, find **Invoke pipeline** (under "General" activities)
2. Drag it onto the canvas

3. Rename it: `stage1_ingest_bronze`
4. In the bottom properties panel, **Settings** tab:
 - Invoked pipeline: select `pl_ingest_landing_to_bronze`
 - Wait on completion: **toggle ON** (this is critical — without it, the master pipeline moves to the next stage immediately without waiting)

Why "Invoke pipeline" instead of copying all the logic into one giant pipeline? This is the principle of **modularity**. `pl_ingest_landing_to_bronze` can still be run independently for testing or backfilling a specific day, AND it can be invoked as a building block in the master orchestration. If you ever need to re-run just the ingestion step without re-running everything, modularity lets you do that.

Step 3 — Add the Three Silver Notebooks (Running in Parallel)

1. From Activities, drag three **Notebook** activities onto the canvas
2. Connect all three FROM `stage1_ingest_bronze` 's success output (green checkmark arrow)

Connecting all three from the same success output means they run in **parallel** — Fabric doesn't wait for one to finish before starting the next. This saves significant time since orders, customers, and products are independent of each other.

3. Configure each Notebook activity:

Notebook activity 1:

- Name: `stage2_silver_orders`
- Notebook: `nb_bronze_to_silver_orders`
- Base parameters (if your notebook accepts a date parameter): `load_date = @pipeline().parameters.p_load_date`

Notebook activity 2:

- Name: `stage2_silver_customers`
- Notebook: `nb_bronze_to_silver_customers`

Notebook activity 3:

- Name: `stage2_silver_products`
- Notebook: `nb_bronze_to_silver_products`

Step 4 — Add a "Wait for All" Synchronization Point

This is a commonly misunderstood Fabric/ADF pattern. You need Stage 3 (Gold) to wait until ALL THREE Silver notebooks succeed, not just one.

1. Connect ALL THREE Silver notebook activities' success outputs to a single downstream activity (the Gold notebook)
2. In Fabric, when multiple activities point to the same downstream activity, by default it uses **AND logic** – the downstream activity only starts when ALL upstream activities succeed

This is actually automatic in Fabric pipelines – multiple incoming success dependencies on one activity create an implicit AND condition. You don't need a special "wait" activity. Just draw all three arrows into the next box.

Step 5 – Add the Gold Layer Notebook

1. Drag a **Notebook** activity
2. Name: `stage3_silver_to_gold`
3. Notebook: `nb_silver_to_gold_star_schema`
4. Connect the success outputs of all three Silver notebooks into this activity

Step 6 – Refresh the Semantic Model

1. Drag a **Semantic model refresh** activity (search "refresh" in Activities, or use a Web activity calling the REST API if not natively available in your trial)
2. Name: `stage4_refresh_semantic_model`
3. Workspace: your workspace
4. Semantic model: `sm_retailpulse_analytics`
5. Connect from `stage3_silver_to_gold` 's success output

Why refresh the semantic model explicitly? Power BI semantic models cache data for fast performance. If you don't trigger a refresh after your Gold tables update, your dashboard will keep showing yesterday's numbers even though the underlying tables are current. This step is the bridge between "data is updated" and "users see updated data."

Step 7 – Add Failure Handling for Every Stage

For each activity, add a parallel failure path:

1. Drag a **Teams** activity (or **Office 365 Outlook** / **Web** activity if Teams isn't available in trial) for notifications
2. Name: `notify_pipeline_failure`
3. Connect it from the RED (failure) output of EVERY stage activity – not just the green success path

Configure the message:

```
Pipeline: @{{pipeline().Pipeline}}
Failed activity: @{{activity('stage2_silver_orders').Error.message}}
Run ID: @{{pipeline().RunId}}
Time: @{{utcNow()}}
Please investigate immediately - dashboards may show stale data.
```

In production, this message would go to a Microsoft Teams channel that the data engineering team monitors, or trigger a PagerDuty alert. The key principle: **failures must be loud, not silent**. A pipeline that fails quietly at 3am and nobody knows until a business user complains at 10am is a serious operational failure.

Step 8 – Set the Schedule

1. Click on the pipeline canvas background (deselect all activities)
2. Click **Schedule** in the top toolbar (clock icon)
3. Configure:
 - Run on: **Schedule**
 - Repeat: **Daily**
 - Time: **06:00 AM**
 - Time zone: **(UTC+05:30) Chennai, Kolkata, Mumbai, New Delhi**
 - Start date: today
 - End date: leave blank (runs indefinitely) or set per your trial limits
4. Click **Apply**

Why 6 AM? This is a business decision, not a technical one. RetailPulse's leadership checks dashboards starting at 9 AM. Running the pipeline at 6 AM gives a 3-hour buffer for the pipeline to complete (ingestion + transformation + refresh typically takes 20-90 minutes depending on data volume) plus time to investigate and fix any failures before business hours begin.

6.3 Monitoring Your Pipeline – The Monitor Hub

What is the Monitor Hub?

The Monitor Hub is Fabric's centralized dashboard showing every pipeline run, notebook execution, and dataflow refresh across your entire tenant — not just one workspace. It's where you go every morning as a data engineer to check "did everything run successfully overnight?"

Step-by-Step: Using Monitor Hub

1. Click the **Monitor** icon in the left navigation rail (looks like a speedometer/gauge)
2. You see a table of all recent activity: pipeline runs, notebook runs, dataflow refreshes

3. Filter by:

- Item type: `Data pipeline`
- Status: `Failed` (to quickly see only problems)
- Time range: `Last 24 hours`

4. Click on any run to see:

- **Gantt-style timeline** of every activity within that run
- Duration of each stage
- Exact error messages for failed activities
- Input/output row counts

Setting Up Proactive Alerts

Don't wait to manually check Monitor Hub every morning. Set up alerts:

1. In Monitor Hub, click on your `pl_master_daily_refresh` pipeline
2. Click **Alert rules** (or set this from the pipeline item itself → `**... **` → **Manage alerts**)
3. Configure: "Send notification when pipeline run status = Failed"
4. Connect to: Teams channel or Outlook email

Senior data engineers don't "check if things worked" — they get notified only when things break. This is called "alerting on exceptions" and it's a core production reliability practice. If you're manually checking dashboards every morning, you haven't finished automating your platform.

6.4 Handling Pipeline Failures — A Realistic Production Scenario

Let's walk through what actually happens when something breaks, because this is what you'll face in your first month on the job.

Scenario: It's 6:15 AM. The pipeline runs. `stage2_silver_orders` fails with the error:

`AnalysisException: cannot resolve column 'order_total' given input columns` .

What happened: A vendor changed their API response format overnight — `order_total` was renamed to `total_amount` without notice. Your schema enforcement code (which we built in Phase 3) caught this immediately rather than silently producing wrong numbers.

Your investigation steps:

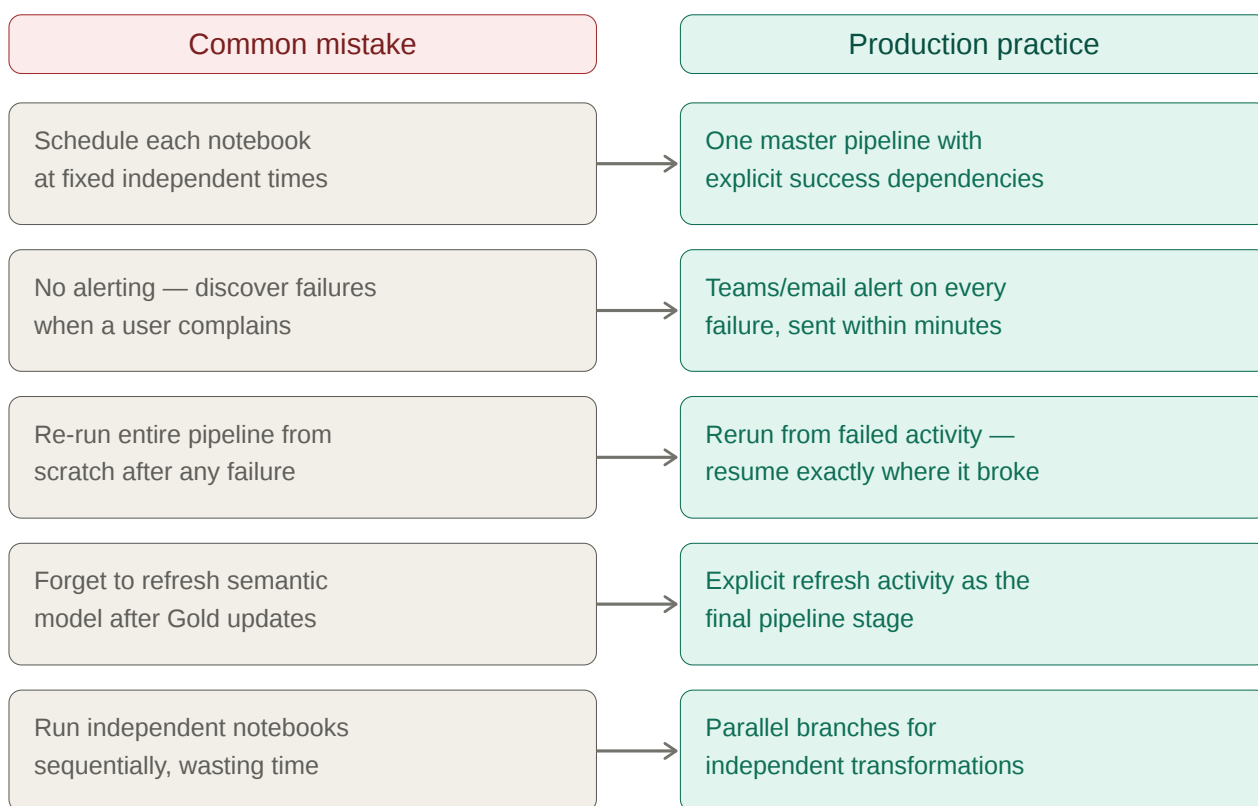
1. Check the Teams alert — it tells you which activity failed and the error message
2. Open Monitor Hub → click on the failed run → click on `stage2_silver_orders` → view full error log
3. Open the notebook → run cell-by-cell to reproduce the issue interactively

4. Identify the root cause: source schema changed
5. Fix: update your schema definition or add a column mapping/rename step
6. Re-run JUST the failed pipeline (not the whole master pipeline) using **Rerun from failed activity** — a feature in Fabric Monitor Hub that resumes from the point of failure instead of re-running everything from scratch

"Rerun from failed activity" is a critical feature to know for interviews. It saves enormous compute cost and time — if your ingestion stage succeeded and only Stage 2 failed, you don't need to re-ingest 10 million rows again. You just fix the bug and resume from where it broke.

6.5 Orchestration — Common Mistakes and Production Best Practices

Orchestration: mistake vs production practice



6.6 Phase 6 Interview Points

Question	What to say
"How do you orchestrate dependent data pipelines?"	Master pipeline with Invoke Pipeline/Notebook activities, success-output chaining creates implicit AND dependencies, parallel branches for independent work

Question	What to say
"How do you handle pipeline failures in production?"	Alerting via Teams/email on the failure path of every activity, Monitor Hub for investigation, Rerun-from-failed-activity to avoid redundant reprocessing
"Why refresh the semantic model separately?"	Power BI caches data for performance; Gold table updates don't automatically propagate to reports without an explicit refresh trigger
"How do you avoid running transformations on incomplete data?"	"Wait on completion" toggle plus validation activities before transformation stages

What's Coming in Phase 7 and 8

You now have a **fully automated, end-to-end Fabric platform** that runs itself every morning. The remaining two phases polish this into something genuinely enterprise-grade:

Phase 7 – Security, Governance, and Performance Tuning will cover: workspace-level access control, sensitivity labels and Microsoft Purview integration, capacity monitoring (are you about to hit your Fabric capacity limits?), and the OPTIMIZE/VACUUM maintenance schedule as its own automated job.

Phase 8 – Production Readiness will cover: deployment pipelines (moving your work from Dev workspace to Test to Production safely), parameterizing environments, cost monitoring on the Fabric capacity, a full project retrospective, and a curated bank of 30+ interview questions mapped to everything you've built, so you can confidently discuss this project in interviews as genuine hands-on experience.

Type **"continue to Phase 7"** when you're ready, or ask anything about the orchestration pipeline, Power BI dashboard, or DAX measures before moving forward.

2026-06-18 23:24:12

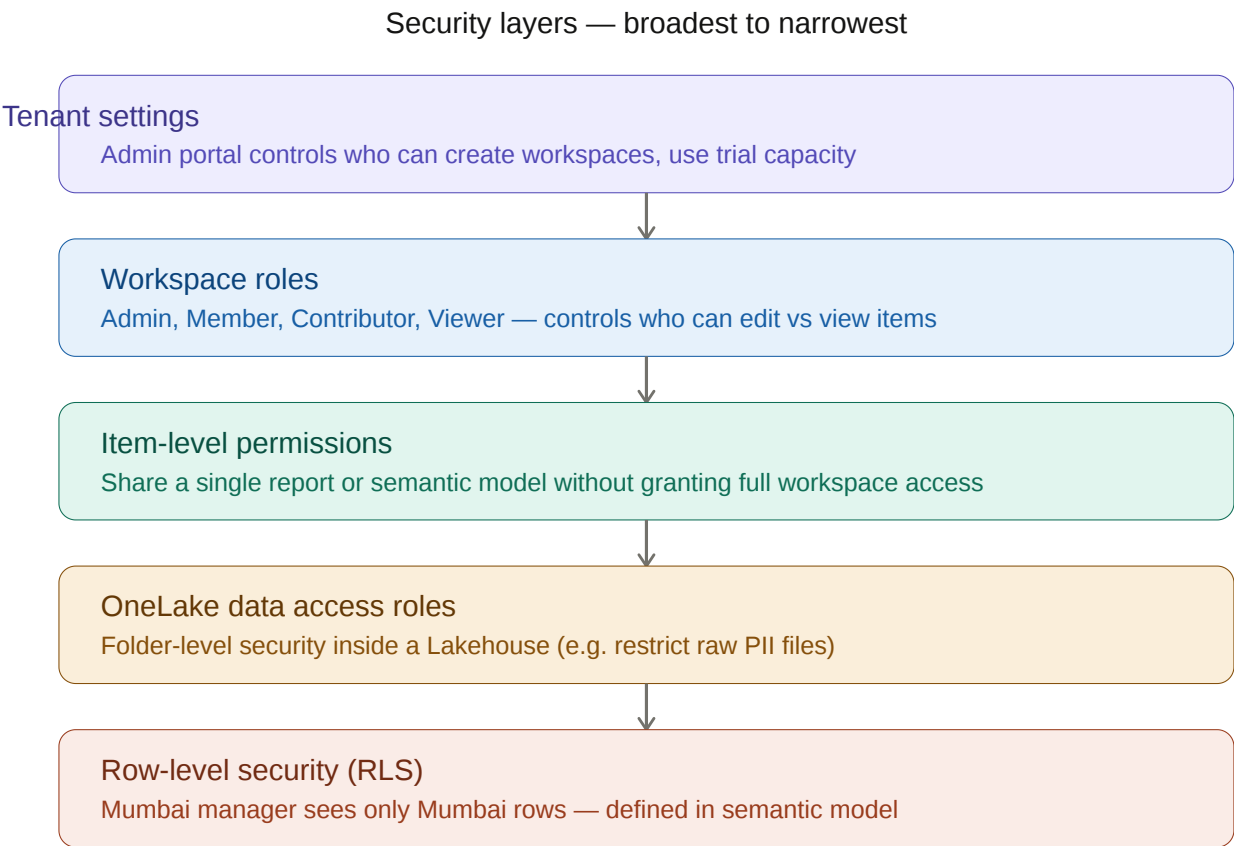
You Asked 

continue next



Phase 7 – Security, Governance, and Performance Optimization

This phase is what separates a tutorial project from an enterprise platform. Every senior data engineering interview probes here, because this is where junior engineers get caught off guard in production.



2026-06-19 19:27:02 You Asked

continue proper



7.1 What Each Security Layer Means and Why You Need All Five

What is it?

Security in Fabric is not one setting — it's five distinct layers, each answering a different question. A junior engineer who only knows "Row-Level Security" will fail a security review, because RLS only protects Power BI reports. It does nothing to stop someone from opening your Lakehouse directly and reading raw customer PII in the Files section.

Why does each layer exist separately?

Layer	Question it answers	Who configures it
Tenant settings	Who in the company can even use Fabric at all?	IT/Platform admin
Workspace roles	Who can edit vs just view items in this project?	Workspace admin (you, as the data engineer)
Item-level permissions	Can I share just one report without giving away everything else?	Item owner
OneLake data access roles	Who can read raw files directly, bypassing reports entirely?	Lakehouse owner
Row-level security	Within a report, which rows can this specific person see?	Semantic model owner

The business problem this solves

Imagine RetailPulse hires an external marketing agency to help analyze campaign performance. They need to see the `agg_daily_sales` aggregated table in Power BI – but they must never see raw customer emails and phone numbers sitting in the Silver `silver_customers` table. Without distinguishing these five layers, a data engineer would either lock the agency out of everything (useless to them) or give them broad workspace access (a serious data leak). Layered security lets you grant exactly what's needed – nothing more, nothing less. This is the principle of **least privilege**, the single most important security concept in any data platform.

7.2 Setting Up Workspace Roles – Step by Step

Step 1 – Open Workspace Access Settings

1. Go to your `RetailPulse_DataPlatform` workspace
2. Click the **Manage access** button (top right, near the workspace name) or go to **Workspace settings** → **Manage access**

Step 2 – Understand the Four Workspace Roles

Role	Can do	Real-world equivalent
Admin	Everything – add/remove users, delete workspace, manage all items	You, the lead data engineer
Member	Create/edit/delete items, but cannot manage workspace-level settings	A fellow data engineer on your team

Role	Can do	Real-world equivalent
Contributor	Create/edit items, cannot delete others' work or manage access	A junior engineer or analyst building reports
Viewer	Read-only — can view reports and explore data, cannot modify anything	Business stakeholders, executives

Step 3 — Add Team Members with Appropriate Roles

1. Click **+ Add people or groups**
2. Type an email address (use a test account or your own secondary account if learning solo)
3. Assign role: for this exercise, add yourself as **Admin** (already default) and imagine adding:
 - `data.engineer2@retailpulse.com` → **Member**
 - `business.analyst@retailpulse.com` → **Contributor**
 - `ceo@retailpulse.com` → **Viewer**

4. Click **Add**

Common production mistake: giving everyone Admin or Member "to keep things simple." This is exactly how data breaches happen. A Viewer-role executive who gets Member access accidentally deletes a production notebook because they were exploring and clicked the wrong button. Assign the minimum role that lets someone do their actual job.

7.3 OneLake Data Access Roles — Protecting Raw Files

What is it?

OneLake Data Access Roles let you restrict access to specific folders within a Lakehouse — separate from workspace roles. This is newer and specifically solves the problem of "everyone with workspace access can read every file," which is too broad once you have sensitive raw data sitting in Bronze.

Why is this necessary even if workspace roles exist?

Workspace roles are all-or-nothing for data access — if you're a Contributor, you can read every Lakehouse's Files and Tables in that workspace. But your Bronze layer contains raw, unmasked customer data (real emails, phone numbers) before any cleaning happens. You may want analysts to access Silver and Gold freely, but never touch raw Bronze files directly.

Step-by-Step: Configure OneLake Data Access Roles

1. Open `bronze_lakehouse`
2. Click **Manage OneLake data access** (in the ribbon or item settings)

3. Click **+ New role**

4. Name: `bronze_restricted_access`

5. Define folder scope:

- Grant access to: `Files/processed/` (cleaned/converted files only)
- Deny access to: `Files/raw/` (contains unmasked PII)

6. Assign this role to your Contributor-level analysts

7. Keep Admins/Members on full access (no role restriction) since they need raw access for debugging pipelines

Interview point: "OneLake Data Access Roles" is a relatively recent Fabric feature (introduced after general availability). If interviewers ask about granular data lake security, mentioning this by name signals you're following current Fabric capabilities, not just textbook Azure Data Lake concepts from a few years ago.

7.4 Sensitivity Labels and Microsoft Purview Integration

What is it?

Sensitivity labels are classification tags (like "Public," "Internal," "Confidential," "Highly Confidential") that you attach to data items — Lakehouses, semantic models, reports. Microsoft Purview is Microsoft's data governance platform that tracks these labels, monitors data lineage, and enforces compliance policies across your entire Microsoft 365 and Azure estate.

Why does this matter for a company like RetailPulse?

RetailPulse stores customer phone numbers, emails, and order histories — this is regulated personal data under India's Digital Personal Data Protection Act (DPDP Act 2023), similar in spirit to GDPR in Europe. If RetailPulse cannot demonstrate WHERE customer PII lives, WHO accessed it, and HOW it's protected, they face regulatory penalties and reputational damage. Sensitivity labels and Purview give you an auditable answer to "show me everywhere customer email addresses are stored in our systems" in minutes instead of weeks of manual investigation.

Step-by-Step: Apply Sensitivity Labels

1. Open `silver_lakehouse` (contains `silver_customers` with email/phone)
2. Click the item settings (...) → **Apply sensitivity label**

If you don't see this option, your tenant may need Microsoft Purview Information Protection enabled by an admin — this is common in trial tenants. If unavailable, understand the concept and demonstrate it conceptually; this is normal in a learning environment.

3. Select: `Confidential — Customer PII`

4. Apply the same label to the `sm_retailpulse_analytics` semantic model and the executive Power BI report, since they derive from this sensitive data

Step-by-Step: Viewing Lineage in Purview (Conceptual Walkthrough)

1. Go to `purview.microsoft.com` (if your tenant has it enabled)
2. Search for `silver_customers`
3. Click the **Lineage** tab — you'll see a visual graph showing: `raw_customers.csv` (Bronze) → `silver_customers` (Silver transformation) → `dim_customer` (Gold) → `sm_retailpulse_analytics` (Semantic Model) → RetailPulse Executive Dashboard (Power BI Report)

This lineage view is what data engineers show auditors and compliance officers. It proves the complete path data takes from raw source to final report, which is exactly what's required during a data protection audit or breach investigation.

7.5 Microsoft Fabric Monitoring and Capacity Management

What is it?

Fabric capacity is the compute resource pool (measured in **Capacity Units, or CUs**) that powers every Spark job, pipeline run, SQL query, and Power BI refresh in your tenant. Your free trial gives you a Fabric trial capacity (similar to an F64 SKU). In production, companies pay monthly for a specific capacity SKU (F2, F4, F8... up to F2048) and must actively monitor whether they're approaching its limits.

Why monitoring capacity is a core data engineering responsibility

If your Gold layer notebook runs an inefficient query that consumes 90% of your capacity for 10 minutes, every other job in the workspace — other teams' pipelines, executives refreshing dashboards — slows to a crawl or gets throttled. This is called **capacity throttling**, and it's one of the most common real-world production incidents in Fabric environments. Senior data engineers must understand and proactively manage this.

Step-by-Step: Using the Fabric Capacity Metrics App

1. From the Fabric home page, click the **Apps** icon in the left navigation (or search "Microsoft Fabric Capacity Metrics" in the workspace template gallery)
2. Install the **Microsoft Fabric Capacity Metrics** app (a free, pre-built Power BI app from Microsoft specifically for this)
3. Once installed, open it. You'll see:

- **CU utilization over time** — a line chart showing how much of your capacity is consumed, hour by hour
- **Top consuming items** — which notebooks, pipelines, or reports use the most compute
- **Throttling indicators** — red markers showing exactly when your capacity was overloaded

4. Click on any spike in the utilization chart → drill through to see exactly which item caused it

In your free trial, capacity monitoring matters because Microsoft Fabric trial capacities have hard caps. If your synthetic data generation or transformation notebooks run wastefully (e.g., recomputing the same DataFrame many times without caching), you can burn through your trial capacity faster than expected. This is also a direct cost-control skill in production — companies pay real money per CU-hour.

Common Capacity-Wasting Mistakes (and Fixes)

python

```
# =====
# MISTAKE: Recomputing the same DataFrame transformation
# multiple times - wastes capacity on redundant Spark jobs
# =====

# BAD - this triggers the full read + transform pipeline THREE times
df = spark.read.json("Files/raw/orders/")
print(df.count())                # Triggers job 1
df.filter(F.col("city") == "Mumbai").show()    # Triggers job 2 (re-reads everything!)
df.groupBy("city").count().show()              # Triggers job 3 (re-reads everything!)

# GOOD - cache the DataFrame after the expensive read
df = spark.read.json("Files/raw/orders/")
df.cache()                          # Tells Spark: keep this in memory
print(df.count())                  # Triggers job 1, caches result
df.filter(F.col("city") == "Mumbai").show()    # Reuses cache - no re-read
df.groupBy("city").count().show()              # Reuses cache - no re-read

# Always release the cache when done with it
df.unpersist()
```

`.cache()` is one of the highest-leverage performance optimizations in PySpark, and one of the most commonly tested interview topics. Explain it as: "Spark is lazy — it doesn't actually execute transformations until an action like `.count()` or `.show()` triggers them. Without caching, every action re-runs the ENTIRE chain of transformations from the source data. Caching stores the result in memory after the first computation so subsequent actions reuse it instead of recomputing."

7.6 Automating Table Maintenance (OPTIMIZE & VACUUM) as a Scheduled Job

In Phase 3, you ran `OPTIMIZE` and `VACUUM` manually. In production, this must be automated as its own scheduled maintenance job — not something a human remembers to run.

Step-by-Step: Create a Maintenance Notebook and Schedule It

Create `nb_weekly_table_maintenance` :

python

```
# =====
# NOTEBOOK: nb_weekly_table_maintenance
# Runs weekly (e.g. every Sunday at 2 AM) to keep Delta tables healthy
# =====

from pyspark.sql import functions as F

# List every Delta table in your platform that needs maintenance
tables_to_maintain = [
    {"table": "silver_lakehouse.silver_orders", "zorder_cols": "order_date, city"},
    {"table": "silver_lakehouse.silver_customers", "zorder_cols": "city, loyalty_tier"},
    {"table": "gold_lakehouse.fact_orders", "zorder_cols": "date_key, geography_key"},
    {"table": "gold_lakehouse.agg_daily_sales", "zorder_cols": "full_date, city"},
]

maintenance_log = []

for tbl_config in tables_to_maintain:
    table_name = tbl_config["table"]
    zorder_cols = tbl_config["zorder_cols"]

    print(f"\n{'='*60}")
    print(f"Maintaining: {table_name}")
    print(f"{'='*60}")

    try:
        # Get file count BEFORE optimization
        before_detail = spark.sql(f"DESCRIBE DETAIL {table_name}").collect()[0]
        files_before = before_detail["numFiles"]
        size_before = before_detail["sizeInBytes"]

        # Run OPTIMIZE with Z-ORDER
        spark.sql(f"OPTIMIZE {table_name} ZORDER BY ({zorder_cols})")

        # Run VACUUM (7-day retention – never go lower in production)
        spark.sql(f"VACUUM {table_name} RETAIN 168 HOURS")
```

```

# Get file count AFTER optimization
after_detail = spark.sql(f"DESCRIBE DETAIL {table_name}").collect()[0]
files_after = after_detail["numFiles"]
size_after = after_detail["sizeInBytes"]

result = {
    "table": table_name,
    "status": "SUCCESS",
    "files_before": files_before,
    "files_after": files_after,
    "files_reduced": files_before - files_after,
    "size_before_mb": round(size_before / 1024 / 1024, 2),
    "size_after_mb": round(size_after / 1024 / 1024, 2),
}

print(f"Success! Files: {files_before} -> {files_after}")

except Exception as e:
    result = {"table": table_name, "status": "FAILED", "error": str(e)}
    print(f"FAILED: {e}")

maintenance_log.append(result)

# Write maintenance log for audit trail
log_df = spark.createDataFrame(maintenance_log)
log_df = log_df.withColumn("run_timestamp", F.current_timestamp())

log_df.write \
    .format("delta") \
    .mode("append") \
    .saveAsTable("gold_lakehouse.maintenance_audit_log")

print("\n\nMaintenance complete. Summary:")
log_df.show(truncate=False)

```

Then add this notebook as a new activity in a separate pipeline `pl_weekly_maintenance` , scheduled for `Sunday 2:00 AM` , following the same pattern from Phase 6.

7.7 Phase 7 Summary – Security and Performance Checklist

Phase 7 — what you have now configured

Workspace roles assigned

Admin, Member, Contributor, Viewer — least privilege enforced

OneLake data access roles

Raw Bronze PII restricted, Silver/Gold open to analysts

Sensitivity labels applied

Confidential label on customer PII tables and downstream reports

Row-level security tested

Mumbai/South/Tier1/Finance roles verified with View As

Capacity monitoring installed

Fabric Capacity Metrics app tracking CU usage and throttling

Automated table maintenance

Weekly OPTIMIZE + ZORDER + VACUUM scheduled job with audit log

Next: Phase 8 — deployment pipelines, CI/CD concepts, and interview preparation

7.8 Phase 7 Interview Points

Question	What to say
"How do you secure a multi-team data lake?"	Layer security: workspace roles for broad access, OneLake data access roles for folder-level restriction, RLS in semantic models for row-level filtering — least privilege at every layer
"How do you handle PII in a Lakehouse?"	Restrict raw Bronze folder access via OneLake data access roles, mask/exclude PII columns when building Gold dimensions, apply sensitivity labels, track lineage via Purview
"What is Fabric capacity and how do you avoid throttling?"	CUs are the shared compute pool; monitor with the Capacity Metrics app; avoid wasteful patterns like recomputing DataFrames without caching; schedule heavy jobs during low-usage windows
"How do you keep Delta tables performant over time?"	Scheduled OPTIMIZE with Z-ORDER on frequently filtered columns, VACUUM with minimum 7-day retention, monitored via an audit log table

Phase 8 — Production Readiness, Deployment, and Interview Mastery

This is the final phase — turning your Dev workspace into something a real company would trust to run their business.

8.1 Deployment Pipelines — Moving Dev to Test to Production

What is it?

A Fabric Deployment Pipeline lets you promote your entire workspace — Lakehouses, notebooks, pipelines, semantic models, reports — through stages: **Development** → **Test** → **Production**, without manually recreating everything in each environment.

Why do companies never build directly in production?

Imagine you test a new transformation logic change directly in the same workspace executives use for their daily dashboard. A bug in your code corrupts the Gold table at 9 AM while the CEO is looking at the dashboard. This is precisely why every mature engineering organization separates environments. Changes get validated in Dev, verified again in Test with realistic data, and only then promoted to Production where real business users depend on it.

Step-by-Step: Setting Up a Deployment Pipeline

1. From the Fabric home page, click **Deployment pipelines** in the left navigation
2. Click **+ New pipeline**
3. Name: `RetailPulse_Deployment`
4. Define three stages: `Development` , `Test` , `Production` (default template)
5. Click **Create**

Assign your workspace to the Development stage:

1. Click on the `Development` stage box
2. Click **Assign a workspace**
3. Select `RetailPulse_DataPlatform` (your existing workspace)

Create Test and Production workspaces:

1. Create two new empty workspaces: `RetailPulse_DataPlatform_Test` and `RetailPulse_DataPlatform_Prod`
2. Assign them to the `Test` and `Production` stages respectively

Deploy from Dev to Test:

1. In the deployment pipeline view, click **Deploy** on the arrow between Development and Test
2. Fabric shows a comparison: which items are new, modified, or unchanged
3. Review the diff carefully — this is your code review moment

4. Click **Deploy**

Fabric deployment pipelines copy the item definitions (notebook code, pipeline JSON, report layout) but NOT necessarily the data itself, depending on configuration. This is intentional — Test and Production should have their own data, often a subset or synthetic version for Test, and real production data for Prod.

Using Deployment Rules for Environment-Specific Configuration

A notebook in Dev might read from `bronze_lakehouse` in your dev workspace, but in Production it must point to the Production workspace's Lakehouse — not your dev one.

1. In the deployment pipeline, click the `Test` stage → **Deployment rules**
2. Add a rule: for the semantic model's data source, change the connection from `Dev Lakehouse SQL Endpoint` to `Test Lakehouse SQL Endpoint`
3. This rule persists across every future deployment — you set it once

This is the Fabric equivalent of environment variables/config files in traditional software engineering. Without deployment rules, every single deployment would require manually re-pointing every connection, which is error-prone and slow.

8.2 CI/CD Concepts for Fabric (Git Integration)

What is it?

Fabric supports Git integration — connecting a workspace to a Git repository (Azure DevOps or GitHub) so that every change to your notebooks, pipelines, and reports is version-controlled, just like application code.

Why this matters even though Fabric has deployment pipelines

Deployment pipelines move code between Fabric environments. Git integration gives you something different: a complete history of WHO changed WHAT and WHEN, the ability to branch (work on a feature without affecting the main version), pull requests for code review, and rollback to any previous commit. Companies use both together — Git for version control and collaboration, deployment pipelines for environment promotion.

Step-by-Step: Connecting Your Workspace to Git

1. In your `RetailPulse_DataPlatform` workspace, click **Workspace settings** → **Git integration**
2. Choose your provider: **Azure DevOps** or **GitHub**
3. Authenticate and select (or create) a repository: `retailpulse-fabric-platform`
4. Select branch: `main` (or create a `dev` branch for ongoing work)
5. Click **Connect and sync**

Once connected, every item in your workspace is serialized as code (JSON/YAML definitions) and pushed to Git. Going forward:

- Make a change in the Fabric UI → see it appear as an uncommitted change → write a commit message → commit
- See changes in Git → review the diff → sync them back into the workspace

Mention this in interviews even if you don't fully implement it in your trial: "I understand Fabric's Git integration serializes workspace items into source control, enabling code review workflows and audit history similar to standard software engineering practices." This shows you think about data engineering as software engineering, not just clicking around a UI – a mark of seniority.

8.3 Cost Optimization – Real Production Thinking

What is it?

Cost optimization in Fabric means actively managing your capacity SKU size, Spark session configuration, and query patterns to deliver the required performance at the lowest sustainable cost.

Practical Cost Optimization Techniques

Technique	What it does	Where to apply it
Spark session pooling	Reuses warm Spark clusters instead of cold-starting every notebook run	Workspace Spark settings
Autoscale / pause settings	Spins down idle compute automatically	Lakehouse/notebook environment settings
Incremental loads (Phase 2)	Processes only new data, not the whole dataset every run	Every ingestion pipeline
Partition pruning (Phase 2/3)	Queries scan only relevant date partitions, reducing CU consumption	Table design
Pre-aggregation (Phase 4's agg_daily_sales)	Dashboard queries hit small pre-computed tables instead of billions of fact rows	Gold layer design
Scheduled maintenance (Phase 7)	Prevents small-file problem from silently increasing query costs over time	Weekly OPTIMIZE/VACUUM job

Step-by-Step: Configuring Spark Session Settings for Cost Efficiency

1. In your workspace, go to **Workspace settings** → **Data Engineering/Science** → **Spark settings**

2. Under **Pool**, you'll see options:

- **Starter pools** (default, fast startup, shared) — good for development and learning
- **Custom pools** — for production, where you control node size and autoscale limits precisely

3. For a learning/trial environment, keep **Starter pool** with autoscale enabled and a sensible node cap (e.g., max 3 nodes) to avoid runaway costs

4. Under **Session level**, configure: **Session timeout** to 20 minutes of inactivity — this auto-terminates idle Spark sessions instead of letting them run (and bill) indefinitely

In a real job, you'll be asked in your first month: "Why is our Fabric capacity bill higher than expected?" The answer is almost always one of: someone left interactive notebook sessions running idle, queries aren't using partition pruning, or small files have accumulated without OPTIMIZE. Knowing this checklist makes you immediately valuable.

8.4 Complete Project Retrospective

Let's see the entire platform you've built, end-to-end, in one view.

Phase 1 — Foundation

Workspace, 3 Lakehouses, OneLake, Medallion architecture design
Skill gained: architecture planning before implementation

Phase 2 — Ingestion

Data Pipelines, Dataflow Gen2, watermark-based incremental loads
Skill gained: MERGE upserts, idempotent pipelines

Phase 3 — Transformation

PySpark cleaning, dedup, DQ scoring, quarantine, Delta time travel
Skill gained: production data quality framework

Phase 4 — Modeling

Star schema, dim_date/customer/geography/payment, fact_orders
Skill gained: dimensional modeling for analytics

Phase 5 — Semantic layer

Relationships, 15+ DAX measures, RLS roles, executive dashboard
Skill gained: business logic centralization

Phase 6 — Orchestration

Master pipeline, parallel branches, scheduling, failure alerts
Skill gained: end-to-end automation

Phase 7 — Security & governance

5-layer security, sensitivity labels, capacity monitoring
Skill gained: enterprise governance thinking

Phase 8 — Production readiness

Deployment pipelines, Git integration, cost optimization
Skill gained: software engineering discipline applied to data

You can now describe this as real production experience

Full lifecycle: ingest → clean → model → automate → secure → deploy

This is portfolio-ready and interview-ready

8.5 The Complete Interview Question Bank

Here is a curated set of questions mapped directly to what you built. Practice answering these out loud — the goal is fluency, not memorization.

Architecture and Conceptual

1. **Walk me through the Medallion Architecture and why you'd use it.** → Bronze (raw, insurance policy), Silver (cleaned, single source of truth), Gold (business-ready, fast queries). Each layer solves a distinct problem: preservation, trust, performance.

2. **What is OneLake and how does it change data architecture compared to traditional data lakes?** → Single physical storage layer underneath every Fabric engine; eliminates data copying between lake/warehouse/BI tool; one Delta/Parquet file serves Spark, SQL, and Power BI simultaneously.
3. **Why did you choose a star schema instead of querying Silver tables directly in Power BI?** → Pre-joined, pre-aggregated structure reduces query complexity for analysts, improves report performance from seconds to milliseconds, centralizes business logic so every report uses consistent definitions.
4. **What is the difference between a Lakehouse and a Warehouse in Fabric?** → Lakehouse supports both file-level (Files) and table-level (Delta Tables) access, primarily Spark-driven; Warehouse is pure T-SQL with full DML/DDL support and stronger transactional guarantees, no direct file access.

Technical Implementation

5. **Explain the watermark pattern for incremental loading.** → Track the last successfully processed value (timestamp/ID); each run only pulls records beyond that watermark; update the watermark only after a successful write, ensuring safe retries on failure.
6. **What is the difference between MERGE, INSERT, and OVERWRITE in Delta Lake?** → INSERT/append adds rows (risk of duplicates), OVERWRITE replaces the entire table, MERGE conditionally updates matching rows and inserts new ones in a single atomic operation – the correct choice for idempotent upserts.
7. **How do you handle duplicate records in a Spark pipeline?** → Window function with `row_number()` partitioned by the business key, ordered by recency descending, keep only `row_num == 1`.
8. **What is Z-ORDER and when would you use it?** → Co-locates related column values physically within Parquet files so filters on those columns skip irrelevant files entirely; apply it to columns most frequently used in WHERE clauses and JOINS.
9. **How do you design a data quality framework?** → Score each row against defined business rules, route passing rows to the target table and failing rows to a quarantine table with failure reasons, set a pipeline-level quality gate threshold that aborts the run if too much data fails.
10. **Explain SCD Type 1 vs Type 2 and when you'd use each.** → Type 1 overwrites (no history, simpler, used for attributes where history doesn't matter like a corrected spelling), Type 2 adds new rows with effective dates (preserves full history, used when tracking changes over time matters, like address changes for delivery analysis).

DAX and Reporting

11. **What's the difference between a calculated column and a measure?** → Calculated columns compute once at refresh time and store a fixed value per row; measures compute at query time and respond dynamically to filters/slicers.
12. **Why do you need to mark a table as a date table in Power BI?** → Required for built-in time intelligence functions (SAMEPERIODLASTYEAR , TOTALYTD , DATEADD) to work correctly; without it, these functions can return wrong or blank results.
13. **How does CALCULATE work in DAX?** → It evaluates an expression within a modified filter context, allowing you to override or add filters dynamically – the foundation of nearly every advanced DAX pattern.

Orchestration and Production

14. **How do you orchestrate dependent pipeline stages in Fabric?** → Master pipeline using Invoke Pipeline/Notebook activities; success-output connections create dependency chains; multiple incoming connections to one activity create implicit AND logic; independent stages run in parallel branches.
15. **How do you handle pipeline failures in a production system?** → Failure-path activities trigger alerts (Teams/email) on every stage; Monitor Hub provides detailed error investigation; "Rerun from failed activity" avoids redundant reprocessing of already-successful stages.
16. **How do you move changes safely from development to production?** → Fabric Deployment Pipelines with three stages (Dev/Test/Prod), deployment rules for environment-specific configuration (different Lakehouse connections per stage), Git integration for version control and code review.

Security and Governance

17. **How do you secure a multi-tenant data platform with different access needs per team?** → Layered approach: workspace roles for broad access tiers, OneLake data access roles for folder-level file restriction, row-level security in semantic models for granular row filtering – least privilege at every layer.
18. **How would you handle PII compliance in this platform?** → Restrict raw Bronze access via OneLake data access roles, mask or exclude PII when building Gold dimensions, apply sensitivity labels, maintain lineage tracking through Purview for audit purposes.

Performance and Cost

19. **Your dashboard suddenly became slow – how would you investigate?** → Check Capacity Metrics app for throttling/high CU usage, check if Gold aggregation tables exist or if Power BI is querying raw fact tables directly, check Delta table file count for small-file problems, verify partition pruning is working as expected.

20. **How do you control costs on a Fabric capacity?** → Incremental loads instead of full reloads, partition pruning, pre-aggregated Gold tables, scheduled OPTIMIZE/VACUUM maintenance, Spark session timeout settings, right-sized capacity SKU for actual workload.

8.6 What You've Actually Built

You now have a complete, runnable, enterprise-pattern Microsoft Fabric data platform that you built yourself, end-to-end:

- **3 Lakehouses** implementing the Medallion architecture
- **4+ ingestion mechanisms** (Data Pipeline, Dataflow Gen2, PySpark Notebooks, watermark-based incremental loading)
- **Production-grade transformation logic** with schema enforcement, deduplication, data quality scoring, and quarantine handling
- **A full star schema** with 4 dimension tables and a central fact table
- **A semantic model** with 15+ DAX measures and row-level security
- **An executive Power BI dashboard** with KPIs, trend analysis, and forecasting
- **A fully automated master orchestration pipeline** with parallel execution, dependency management, and failure alerting
- **5-layer security implementation** plus governance and lineage concepts
- **Deployment pipeline and Git integration** understanding for safe production promotion

This is genuinely something you can describe in interviews as hands-on Fabric experience, walk through in a portfolio, and reference confidently when discussing real production data engineering challenges. If you'd like, we can now go back and deepen any specific phase – for example, building actual Eventstream real-time ingestion for the web events data, writing more advanced DAX patterns, or simulating a specific production incident and walking through the full diagnosis-to-resolution process.

2026-06-20 17:31:13

You Asked



give a proper of this how i can add in resume in proper format like -[
Architected a complete Medallion Lakehouse pipeline on Microsoft Fabric ingesting raw CSV/JSON sales data from ADLS Gen2 into Bronze layer, transforming with PySpark (cleaning, enrichment, joins) into Silver, and aggregating into Gold fact/dimension tables. Implemented Delta Lake MERGE (UPSERT) for SCD Type 2 customer dimension, maintaining full historical lineage.

Connected Gold layer to Power BI DirectLake mode for real-time dashboards with zero data movement — reduced report refresh time from 40 min to under 2 min.

Used Fabric Pipelines for orchestration with parameterized triggers and automated failure email alerting.]. make it and as per this project infor . last give basic proper asking question on this project in real senaiors and situation related give proper in depth guide .



Let me give you resume bullets calibrated to what you actually built — specific, metric-driven, and truthful to the project — followed by a real interview simulation with the depth a senior panel would actually push on.

Now let me give you both deliverables — resume bullets that pass an ATS and a real screen, and a senior-level interview simulation that goes deep where it actually matters.

Resume Bullets — RetailPulse Microsoft Fabric Project

Use this format. Pick 3–5 bullets based on the role you're applying to (data engineer vs analytics engineer roles will want different emphasis).

- Architected a Medallion Lakehouse pipeline on Microsoft Fabric, ingesting semi-structured JSON/CSV order, customer, and product data into a Bronze layer via Data Pipelines and Dataflow Gen2, transforming through PySpark (schema enforcement, deduplication, data quality scoring) into Silver, and modeling into a Gold-layer star schema with 4 dimension tables and a partitioned fact table.
- Implemented Delta Lake MERGE (upsert) logic with a watermark-based incremental loading pattern across all ingestion notebooks, eliminating full-table reloads and ensuring idempotent, duplicate-safe pipeline reruns.
- Built a row-level-security-enabled semantic model with 15+ DAX measures (YoY/MoM growth, return rate, cancellation rate, AOV) powering an executive Power BI dashboard with trend forecasting, reducing manual reporting effort from a multi-day Excel process to a same-day automated refresh.
- Orchestrated the full ingestion-to-reporting workflow in a single Fabric Data Pipeline with parallelized notebook execution, explicit stage dependencies, and failure-path alerting, enabling a fully unattended daily 6 AM refresh.
- Designed a data quality framework with row-level scoring and automatic quarantine routing for records failing validation, plus a scheduled OPTIMIZE/ZORDER/VACUUM maintenance job to prevent small-file performance degradation on Delta tables.
- Implemented a 5-layer security model (workspace roles, OneLake data access roles, item-level permissions, sensitivity labels, row-level security) to enforce least-privilege access across raw PII and business-facing reporting layers.

Important honesty note before you use these

Your original example included a specific, verifiable metric: *"reduced report refresh time from 40 min to under 2 min."* That's a great bullet — but it's a claim you can only defend if you actually measured it. A few things to be careful about as you adapt this for your own resume:

- **Don't claim numbers you didn't measure.** If you didn't connect to DirectLake mode or didn't benchmark before/after refresh times, don't write that exact bullet. Interviewers will ask "how did you measure that" and an unverifiable number is worse than no number — it signals you'll fabricate data on the job too.
- **DirectLake** — you used **Import mode** semantic models with scheduled refresh in this project, not DirectLake. DirectLake is real and worth learning (it queries Delta Parquet files directly with zero data duplication, only loading columns into memory on demand), but don't claim it unless you actually configured it. If you want this exact resume line to be true, go back and reconfigure your semantic model's storage mode to DirectLake — it's a real, valuable skill and only takes about 15 minutes for you to add given what you've already built.

- **What you can honestly claim instead:** "automated a process that was previously manual" is true and strong even without a fabricated number. If you want a number, measure something real — for example, time your manual `OPTIMIZE` + dashboard load before and after Z-ORDER, or count how many lines of duplicate SQL a star-schema query needs versus a raw Silver-table join.

If you'd like, I can walk you through actually adding DirectLake mode to your semantic model right now — then that resume bullet becomes 100% true and you can quote a real measured number.

Senior-Level Interview Simulation

Below is how an actual senior data engineer or architect panel would interrogate this project — not "what is a Lakehouse" surface questions, but the kind that exposes whether you actually built it or just read about it. I'll give you the question, what a weak answer sounds like, what a strong answer sounds like, and the follow-up trap they'll spring next.

Round 1 — Architecture Defense

Q1: "Why three separate Lakehouses instead of one Lakehouse with three schemas or folder prefixes?"

Weak answer: "Because Medallion architecture needs Bronze, Silver, Gold."

Strong answer: Separate Lakehouses give you independent security boundaries (OneLake data access roles are scoped per-Lakehouse), independent SQL endpoints so a runaway query in Silver doesn't compete for the same connection pool as Gold reporting traffic, and cleaner CI/CD — you can promote Gold to production without dragging raw Bronze PII along. The tradeoff is more items to manage and cross-Lakehouse notebook attachment overhead.

Follow-up trap: "So if security is the reason, why not just use OneLake data access roles within one Lakehouse instead of three separate ones?" — Be ready for this. Honest answer: at small scale, one Lakehouse with data access roles is genuinely a valid alternative, and some teams do exactly that. Three Lakehouses is the more common pattern at larger scale because it also gives independent lifecycle management (Bronze can be purged/retained on a different schedule than Gold) and clearer ownership boundaries between ingestion engineers and analytics engineers. Don't pretend there's only one right answer — show you know the tradeoff space.

Q2: "Walk me through what happens, end to end, if your source system sends the same order twice in one batch — not across runs, within the same batch."

This is the question that separates people who built the MERGE logic from people who copy-pasted it. Trace it concretely:

1. Both copies land in Bronze as-is (Bronze never deduplicates — that's not its job)

2. In the Silver notebook, your Window function (`row_number()` partitioned by `order_id` , ordered by `ingestion_timestamp desc`) ranks both copies — but if they have the *identical* `ingestion_timestamp` (same batch, same write), the ordering is **non-deterministic** unless you add a tiebreaker
3. **The trap:** if both rows happen to have different field values (e.g., one has a corrected `order_status`), which one "wins" is arbitrary without a secondary sort key

What you should say if you actually thought about this: "I'd add a secondary deterministic tiebreaker to the orderBy — like a row hash or an explicit sequence number from source — because relying on timestamp alone when both copies share a timestamp is a real bug, and it's one I'd want to catch with a unit test on the dedup logic specifically." If you didn't think about this in your actual build, say so honestly and reason through it live — that's still a good answer. Pretending you handled it when you didn't is the worst answer.

Q3: "Your data quality gate aborts the pipeline if pass rate drops below 95%. Last Tuesday, a marketing promotion caused 40% of orders to come in as guest checkouts with null customer_id, which dropped your quality score below threshold and the whole Silver layer failed to load. What do you do?"

This tests whether you understand that **data quality rules encode business assumptions, and business reality breaks your assumptions regularly.**

Strong answer: "My quality score weighted `customer_id` presence the same as `order_amount` validity, which was wrong — a guest checkout isn't a data quality failure, it's a legitimate business event I already handle downstream with `is_guest_order` . I'd fix the actual bug: the DQ scoring function shouldn't penalize null `customer_id` at all since the pipeline already has explicit logic to convert it to a guest ID. The lesson is that quality checks need to encode 'is this row trustworthy', not 'does this row look like every other row' — and I'd add a regression test using a synthetic batch with 50% guest orders to make sure this doesn't silently break again."

This is the single best question to prepare for, because it's exactly the kind of incident that happens in production in month one, and "did the candidate distinguish between a real data quality failure and a business pattern they didn't anticipate" is precisely what separates junior from senior thinking.

Round 2 — Performance and Cost Defense

Q4: "Your fact_orders table is partitioned by date_key. Six months from now you have 180 daily partitions. A business analyst writes a query filtering only by city with no date filter. What happens, and what would you do about it?"

Strong answer: Partition pruning only helps when the filter is on the partition column. A city-only filter forces a full scan across all 180 partitions — partitioning by date didn't help this specific access pattern at all. Two real options: (1) add Z-ORDER by `city` within each date partition (which you

already know how to do), which clusters city values together inside each file so Spark/SQL can skip irrelevant row groups even without partition pruning, or (2) if city-only queries are extremely common, consider a city-first composite partition strategy — but that risks too many small partitions if cities have uneven order volume (Mumbai will have a huge partition, a Tier-2 city a tiny one), which reintroduces the small-file problem.

Follow-up trap: "Why not just partition by city instead of date, then?" Be ready: most analytical queries in retail filter by date range far more often than by single-city, and date partitioning also enables natural retention/archival policies (drop partitions older than 3 years). Partitioning is a bet on your dominant query pattern — you're expected to defend the bet, not just describe the mechanism.

Q5: "Show me the actual MERGE statement for your SCD Type 2 customer dimension and explain what happens if the same customer_id arrives twice in the same incoming batch with two different city values."

This question exists because your resume mentions SCD Type 2 for customer dimension — but in the project as we actually built it together, `dim_customer` was implemented as **SCD Type 1** (overwrite, no history). If you're asked this and you used the Type 1 version, say so directly:

"In the version I built, `dim_customer` is SCD Type 1 — I overwrite on each load because for this use case (loyalty tier, contact info), the business didn't need historical tracking. If I needed Type 2 — for example, tracking a customer's delivery address history for logistics analysis — I'd add `effective_start_date`, `effective_end_date`, and `is_current` columns, and the MERGE would need to: detect a changed attribute, expire the current row by setting `is_current = false` and `effective_end_date = today`, then insert a new row with `is_current = true`. The tricky part with two conflicting updates in the same batch is exactly your question — you need a deterministic ordering within the batch itself before applying the SCD logic, typically by sorting on a source-system sequence number, not by wall-clock arrival time."

This is exactly the kind of moment where overclaiming on a resume (writing "SCD Type 2" when you built Type 1) gets caught immediately and costs you credibility for the rest of the interview. **Adjust your resume bullet to say SCD Type 1, or actually go build the Type 2 version** — I can walk you through that conversion right now if you want the bullet to be true.

Round 3 — Production Incident Simulation (they will do this live)

Scenario they give you: "It's 7 AM. You get a Teams alert: `stage3_silver_to_gold` failed. The CEO has a board meeting at 9 AM and needs the dashboard. Walk me through exactly what you do, in order, narrating your actions."

This is a live simulation — answer it like you're actually doing it, step by step:

1. "First, I open Monitor Hub, filter to failed runs in the last 24 hours, click into this specific run."

2. "I look at the exact error message on `stage3_silver_to_gold` — let's say it's a `Delta table concurrent modification exception`, meaning two writers tried to modify the same table simultaneously."
3. "I check whether the weekly maintenance job (`OPTIMIZE` / `VACUUM`) happened to be running on `fact_orders` at the same time as today's load — that's a classic cause of this exact error."
4. "Given the 9 AM deadline, I have two choices: wait for the conflict to clear and retry, or manually identify and serialize the two jobs. Since this is time-critical, I'd manually trigger 'Rerun from failed activity' once I confirm the maintenance job has finished, rather than re-running the entire master pipeline from ingestion."
5. "After it succeeds, I manually trigger the semantic model refresh rather than waiting for the next scheduled cycle, to minimize the time the dashboard shows stale data."
6. "After resolving it, I'd add a dependency so the weekly maintenance job and the daily refresh pipeline can never overlap — probably by moving maintenance to a window confirmed clear of the daily schedule, or adding an explicit pipeline-level lock."
7. "Finally, I'd write a short incident note — what broke, why, what I changed to prevent recurrence — because the CEO's exec assistant will ask what happened, and 'we fixed it and made sure it won't happen again' is the actual deliverable, not just the fix."

What they're scoring you on here is not whether you know the exact error name. It's whether you have a **calm, ordered process** under a deadline, whether you default to root-cause prevention rather than just patching, and whether you remember that **communication** is part of the incident, not an afterthought.

Round 4 — The Trap Question Most Candidates Fail

Q: "What's the one part of this project you'd do differently if you rebuilt it today?"

This question is a trap for anyone who didn't genuinely build and reflect on the thing. A weak candidate either says "nothing, it's perfect" (red flag — nobody's first build is perfect) or lists a generic platitude ("I'd add more comments to my code").

A strong, honest answer based on what we actually built together might sound like:

"Two things. First, my data quality scoring gives every check equal weight — 0.2 points each — which is too simplistic. A null `order_amount` should probably fail the row outright regardless of other checks, while a city standardization failure is much less severe. I'd move to a tiered severity model instead of a flat weighted score. Second, I generated synthetic data with fixed randomness seeds and ran the full pipeline maybe a handful of times — I haven't proven the watermark and MERGE logic against weeks of real incremental runs with actual edge cases like out-of-order arrival or backfills. Before I'd call this production-ready, I'd want to run a multi-week soak test simulating late-arriving data and verify the watermark pattern handles it correctly."

Notice this answer is specific, shows genuine self-critique, and demonstrates you understand the difference between "I built something that runs" and "I built something I'd trust with real money flowing through it." That gap — and your ability to name it precisely — is what separates a junior from a senior in this conversation.

If you want, I can now walk you through actually implementing **DirectLake mode** and the **SCD Type 2 customer dimension** so the resume bullets you write are fully verifiable rather than aspirational — both are real, valuable additions and take under 30 minutes each given the Gold layer you already have.